

Ain Shams University
Faculty of Computer and Information Sciences
BIOINFORMATICS PROGRAM



Smart Eyes

By

Arwa Esmat Mostafa ElNahas
Rashida Khaled Ahmed
Osama Menshawy Othman
Ahmed Hossam Hassan
Ahmed Essam Abdelgawad
Wessam Salah Walid

Under Supervision of

Dr Salsbil Amin

Professor in **Computer Science** Department,
Faculty of Computer and Information Sciences,
Ain Shams University

TA Esraa Hamdi

Assistant Lecturer in **Bioinformatics** Department
Faculty of Computer and Information Sciences,
Ain Shams University

Acknowledgement

We would like to express our deepest gratitude to Dr. Salsabil Amin for her invaluable guidance, insightful feedback, and unwavering support throughout the development of this project. Her expertise and encouragement have been instrumental in shaping our research and helping us navigate the challenges we encountered.

We are also deeply thankful to T.A. Esraa Hamdi for her constant assistance, practical advice, and dedication. Her support during the various stages of this project was crucial, and her suggestions significantly improved the quality of our work.

Abstract

English Abstract:

In a world dominated by visual information, individuals with visual impairments often struggle to navigate and engage with their surroundings effectively. Current assistive technologies provide some level of support, yet they frequently fall short in delivering real-time, comprehensive assistance. The "Smart Eyes" project seeks to bridge this gap by creating an innovative wearable device designed as smart glasses. These glasses, equipped with a camera and audio output system, integrate three core functionalities: object detection, face recognition, and Optical Character Recognition (OCR) coupled with Text-to-Speech (TTS). This multifunctional approach empowers users to detect objects, recognize familiar faces, and read text aloud, thereby significantly enhancing their environmental awareness and autonomy.

Smart Eyes employs a combination of cutting-edge technologies, including real-time video processing, deep learning for face recognition, and sophisticated OCR methods for text extraction^{[1][2]}. Users can seamlessly switch between these modes via a user-friendly toggle button, ensuring a smooth and intuitive experience.

The project's results underscore the effectiveness of Smart Eyes in providing accurate and reliable real-time recognition of objects, faces, and text across various environments. This functionality greatly enhances the ability of visually impaired individuals to interact with and interpret their surroundings independently^[3].

The Smart Eyes project delivers a groundbreaking assistive solution that enhances the independence and quality of life for individuals with visual impairments. By leveraging advanced technologies within a practical wearable device, we offer a versatile and powerful tool that addresses key challenges faced by the visually impaired community. Future efforts will aim to refine the system's performance and broaden its capabilities to tackle an even wider array of obstacles encountered by visually impaired users^[4].

Arabic Abstract:

في عالم تهيمن عليه المعلومات البصرية، يواجه الأفراد ذوو الإعاقات البصرية صعوبة في التنقل والتفاعل مع محيطهم بشكل فعال. توفر التقنيات المساعدة الحالية مستوى معينًا من الدعم، لكنها غالبًا ما تقصر في تقديم المساعدة الشاملة في الوقت الفعلي. يسعى مشروع "العيون الذكية" إلى سد هذه الفجوة من خلال إنشاء جهاز مبتكر يمكن ارتداؤه مصمم كمنظارات ذكية. هذه النظارات، المزودة بكاميرا ونظام إخراج صوتي، تدمج ثلاث وظائف أساسية: اكتشاف الأجسام، التعرف على الوجوه، والتعرف الضوئي على الأحرف. يتيح هذا النهج متعدد الوظائف للمستخدمين اكتشاف الأجسام، (TTS) مع تحويل النص إلى كلام (OCR) التعرف على الوجوه المألوفة، وقراءة النصوص بصوت عالٍ، مما يعزز بشكل كبير من وعيهم بالبيئة المحيطة واستقلاليتهم.

يستخدم مشروع "العيون الذكية" مزيجًا من التقنيات المتطورة، بما في ذلك معالجة الفيديو في الوقت الفعلي، والتعلم العميق للتعرف على الوجوه، وطرق التعرف الضوئي المتقدمة لاستخراج النصوص. يمكن للمستخدمين التبديل بسلاسة بين هذه الأوضاع عبر زر تبديل سهل الاستخدام، مما يضمن تجربة سلسة وبديهية.

تؤكد نتائج المشروع على فعالية نظارات "العيون الذكية" في توفير التعرف الدقيق والموثوق على الأجسام، الوجوه، والنصوص في الوقت الفعلي عبر بيانات متنوعة. تعزز هذه الوظائف بشكل كبير من قدرة الأفراد ذوي الإعاقات البصرية على التفاعل مع محيطهم وفهمه بشكل مستقل.

يقدم مشروع "العيون الذكية" حلاً مبتكرًا مساعدًا يعزز من استقلالية ونوعية حياة الأفراد ذوي الإعاقات البصرية. من خلال الاستفادة من التقنيات المتقدمة داخل جهاز عملي يمكن ارتداؤه، نقدم أداة متعددة الاستخدامات وقوية تعالج التحديات الرئيسية التي تواجه مجتمع ذوي الإعاقات البصرية. ستسعى الجهود المستقبلية إلى تحسين أداء النظام وتوسيع قدراته لمواجهة مجموعة أكبر من العقبات التي يواجهها المستخدمون ذوو الإعاقات البصرية.

Table of Contents

Chapter	Page
Abstract	
Table of Contents	
List of Figures	
List of Abbreviations	
1 Introduction	1
-	
1.1 Motivation.....	1
1.2 Problem Definition.....	2
1.3 Objective.....	3
1.4 Document Organization.....	4
2 Background	5
-	
2.1 OCR Model.....	5
2.1.1 EasyOCR.....	6
2.2 Text to Speech (TTS).....	9
2.2.1 Google Text to Speech (gTTS).....	9
2.3 Yolo8 Model.....	10
2.4 OpenCV.....	12
2.5 Face_Reognition Library.....	13
2.6 Literature Review.....	15
2.6.1 Object Detection.....	15
2.6.2 Face Recognition.....	16
2.6.3 Text Recognition.....	17
3 Analysis and design	18
-	
3.1 System Overview.....	18
3.1.1 System Architecture.....	18
3.1.2 Function Requirements.....	19
3.1.3 Non-functional Requirements.....	20

3.1.4	System Users.....	21
3.2	System Analysis & Design.....	23
3.2.1	Use Case Diagram.....	23
3.2.2	Class Diagram.....	24
3.2.3	Sequence Diagram.....	25
4	Implementation	26
-		
4.1	Functions used in OCR & TTS.....	26
4.2	Functions used in Face Recognition.....	29
4.3	Functions used in Object Detection.....	31
4.4	Hardware.....	33
4.4.1	Components Used.....	33
4.4.2	How They are Used.....	36
5	User Manual	41
-		
6	Conclusions and Future Work	49
-		
6.1	Conclusions.....	49
6.2	Future Work.....	51
	References	53

List of Figures

Figure	Page Number
Figure 1.1 Global trends and predictions of numbers of people who are blind or moderately and severely vision impaired, from 1990–2050	2
Figure 2.1 EasyOCR Framework	8
Figure 2.2 gTTS converting text to speech architecture	10
Figure 2.3 YOLOv8 model	10
Figure 2.4 Overview of the YOLOv8 model variants	11
Figure 2.5 Flowchart of the proposed OpenCV algorithm	12
Figure 2.6 Generating and Comparing Face Embeddings in 128-Dimensional Space Using CNN	14
Figure 3.1 System Architecture Diagram	18
Figure 3.2 Use Case Diagram	23
Figure 3.3 Class Diagram	24
Figure 3.4 Sequence Diagram	25
Figure 4.1 ESP32-CAM Development Board	33
Figure 4.2 FTDI Converter (USB to TTL)	33
Figure 4.3 TP4056 Lithium Battery Charger	34
Figure 4.4 18650 Lithium Ion 3.7V Battery	34
Figure 4.5 XL6009 Step-up Power Converter	35
Figure 4.6 The Connection of FTDI Converter to the ESP32-CAM	36
Figure 4.7 The Connection of Battery to Charger	37

Figure 4.8 Adjust Output Voltage	38
Figure 4.9 The Final Circuit of the Hardware Module	39
Figure 5.1 Application's Welcome Screen	41
Figure 5.2 Application's Login Screen	42
Figure 5.3 Application's Signup Screen	43
Figure 5.4 Application's Complete Profile Screen	44
Figure 5.5 Application's Add Relative Screen	45
Figure 5.6 Application's Main Screen to let user choose the desired functionality	46
Figure 5.7 Application's Emergency Screen	47
Figure 5.8 Application's Profile Screen	48
Figure 5.9 Application's Settings Screen	49
Figure 5.10 Application's Edit Profile Screen	50

List of Abbreviations

Abbreviations	What the Abbreviation stands for
API	Application Programming Interface
BGR	Blue Green Red
CAM	Camera
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
ESP	External Services Provider
FTDI	Future Technology Devices International Limited
GPIO	General-Purpose Input/Output
gTTS	Google Text to Speech
GND	Connect the Ground
HMM	Hidden Markov Model
HOG	Histogram of Oriented Gradients
IO	Input/Output
IP	Internet Protocol
K-PCA	Kernel Principal Component Analysis
LBP	Local Binary Pattern
LED	Light Emitting Diode
LSTM	Long Short-Term Memory
MS COCO	Microsoft Common Objects in Context
MTCNN	Multi-Task Cascaded Convolutional Neural Networks
OCR	Object Character Recognition
OpenCV	Open-Source Computer Vision Library
OTSU	Algorithm Named After Nobuyuki Otsu
PCA	Principal Component Analysis

R-CNN Resnet	Region-Based Convolutional Neural Network
RGB	Red Green Blue
RNN	Recurrent Neural Network
SSID	Service Set Identifier
SVM	Support Vector Machine
TTL	Transistor Transistor Logic
TTS	Text to Speech
USB	Universal Serial Bus
VGG	Visual Geometry Group
WHO	World Health Organization
YOLO	You Only Look Once

Chapter 1

Introduction

In the world of helpful technology, there's a special focus on making things easier for people who can't see well. This kind of technology, called assistive technology, includes tools like talking computers, devices that read books aloud, and even smart glasses like our project, "Smart Eyes." It's all about using innovation to make life better and more accessible for those with visual challenges.

In today's visually oriented world, navigating and interacting with the environment can be profoundly challenging for visually impaired individuals. Traditional assistive technologies provide valuable support but often struggle to deliver real-time assistance that truly enhances independence and accessibility. The "Smart Eyes" project seeks to change this situation by creating a wearable device. This innovative device takes the form of smart glasses equipped with a camera to detect objects in the surroundings, read text using Optical Character Recognition (OCR), and transform this text into spoken words using Text-to-Speech (TTS) technology. Additionally, the device incorporates face recognition technology, allowing users to identify and recognize people in their environment. By integrating these advanced functionalities into a single, wearable solution, Smart Eyes aims to give visually impaired individuals new abilities to understand and move around their surroundings more independently.

1.2 Motivation

There is a condition worse than blindness: having sight but no vision. Our team visited "Dialogue in the Dark"¹ a place where we experienced a glimpse of a day in the life of a blind person. We navigated streets, sat, and ate in public, all alone. It was incredibly challenging for us. To our surprise, when we exited, we discovered that our guide through this darkness was a blind man. We were amazed and deeply touched.

We only spent half an hour experiencing their reality, and it was unsettling to rely on someone other than ourselves for guidance. This brief yet profound experience made us realize the immense challenges faced by visually impaired individuals daily. We could only imagine how they must feel with years passing by, seeing nothing but darkness, and knowing no more than what they are told.

¹ You can visit their website for further information: <https://didegypt.com>

This realization was the turning point for us. It made us understand what we wanted to achieve with our graduation project. It also made us more thankful for the blessings we have. Driven by this vision, and equipped with the knowledge and skills, we decided to create something meaningful. We chose to be their eyes in the darkness, using our expertise to develop technology that can truly make a difference in their lives.

1.2 Problem Definition

Visually impaired individuals face many challenges in their daily lives. They have trouble reading, moving around, recognizing objects, and identifying people. These difficulties can make them less independent and affect their quality of life.

Although current assistive technologies help, they often don't provide the instant, smooth support that is really needed. According to the World Health Organization (WHO), about 2.2 billion people worldwide have some form of visual impairment or blindness. This large number shows how important it is to find better solutions to help visually impaired individuals live more easily and confidently.

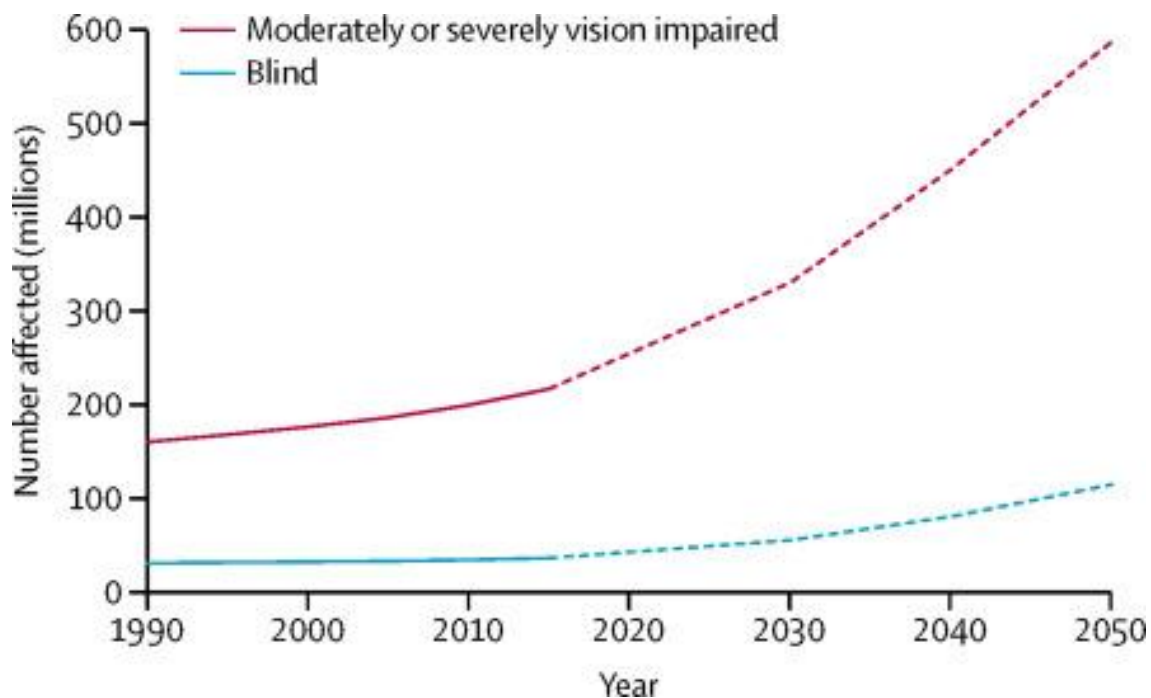


Figure 1.1 Global trends and predictions of numbers of people who are blind or moderately and severely vision impaired, from 1990–2050

1.3 Objective

- **Wearable Device:** Integrate advanced technologies into a wearable device to provide practical and accessible assistance for visually impaired individuals.
- **Text Recognition:** Enable visually impaired individuals to read printed text by using OCR technology.
- **Object Recognition:** Implement an object recognition model to help visually impaired individuals identify objects in their surroundings.
- **Face Recognition:** Develop facial recognition algorithms to identify and recognize familiar faces, such as relatives and family members.
- **Text-to-Speech (TTS):** Convert recognized text into audible content, offering users real-time auditory information.

1.4 Document Organization

Chapter 2: Background

This chapter includes background information about the project's basic concepts and models.

Chapter 3: Analysis and Design

This chapter includes an overview of the whole system along with system functionalities, architecture, and states.

Chapter 4: Implementation

This chapter includes information about all the datasets used in the project.

Chapter 5: User Manual

This chapter shows how a user can use the application and how to use all application features, starting from uploading the chart image until the summary is generated for the user.

Chapter 6: Conclusion and Future Work

This chapter includes the conclusion of the experiments done, the best results, the problems faced during the project, and future work that improves performance.

References

The papers, books, and links that were used in the survey and implementation.

Chapter 2

Background

2.1 OCR Models:

Optical Character Recognition (OCR) technology converts printed or handwritten text from images into machine-readable text, enabling automated document processing and accessibility enhancements. OCR models vary in their approaches and capabilities, each offering unique strengths in text recognition:

- **Tesseract:** Known for robust performance with high-resolution images, benefiting from morphological operations like dilation, erosion, and OTSU binarization.
- **EasyOCR:** A lightweight model that excels in tasks such as receipt or PDF conversion, providing accurate results with organized texts like PDF files, receipts, and bills.
- **OCRopus:** system designed for accurate text recognition from scanned documents, using a blend of machine learning and traditional methods.
- **Kraken:** employs deep learning techniques to identify text in scanned images, supporting multiple languages and document formats.
- **Adobe OCR:** Adobe's tool in Acrobat that extracts text from scanned documents and images, integrated within Adobe's suite for effective document management and text extraction.

OCR models like Tesseract, EasyOCR, OCRopus, Kraken, and Adobe OCR leverage a combination of traditional techniques and deep learning advancements to meet diverse needs in text recognition. These models continue to evolve, driven by advancements in machine learning, improving the efficiency and accuracy of OCR technology across various industries and applications.

2.1.1 EasyOCR:

For our project, we chose to implement EasyOCR due to its efficiency and suitability for our specific needs.

EasyOCR is an open-source Optical Character Recognition (OCR) tool designed to efficiently detect and recognize text in images. Developed with a focus on high accuracy and versatility, it is suitable for deployment on various devices and applications, including document digitization and accessibility tools. EasyOCR leverages advanced deep learning techniques to achieve state-of-the-art performance in text recognition tasks while maintaining computational efficiency.

It achieves high accuracy across a variety of text recognition tasks, supporting multiple languages and scripts. The key innovation of EasyOCR lies in its use of deep learning models as VGG & ResNet to enhance text detection and recognition accuracy. The tool's architecture effectively combines text detection and recognition into a cohesive pipeline, optimizing performance for real-world applications.

EasyOCR consists of three components, The three main components of EasyOCR, are feature extraction, sequence labeling, and decoding. In order to extract useful features from the input image, deep learning models such as ResNet and VGG are used in feature extraction. These characteristics are essential for text recognition in pictures. Sequence labeling uses Long Short-Term Memory (LSTM) networks to interpret the extracted features' sequential context. Text pattern recognition and structuring are crucial tasks for LSTM networks. Finally, the decoding part decodes and transcribes the labeled sequences into the actual recognized text using the Connectionist Temporal Classification (CTC) algorithm. These three elements function as a unit to allow EasyOCR to reliably and effectively extract text from images. The training pipeline is based on the deep-text-recognition-benchmark framework, which enhances text recognition in images and offers a strong basis for OCR execution.

- **Feature Extraction (Resnet and VGG):**

The recognition model's first step is feature extraction. To create a set of features that can be utilized for additional analysis, the input data must be converted. With EasyOCR, VGG and Resnet are used for this.

Resnet, is a kind of convolutional neural network (CNN) that bypasses certain layers by using shortcuts or skip connections. Learning the residual representation functions rather than the signal representation directly is the fundamental tenet of the Resnet architecture. This lowers the model's complexity and facilitates the network's learning process.

An additional kind of CNN is called Visual Geometry Group, or VGG. Its uniform architecture and simplicity are well-known. Its homogeneous architecture consists of very small (3x3) convolution filters stacked deeper and deeper on top of one another. This simplifies the network's understanding and modification by lowering the number of computation and parameters.

- **Sequence Labeling (LSTM):**

Recurrent neural networks (RNNs) of the long-sequence learning and memory type, or LSTMs, are able to model the temporal dependencies of multiple time steps. The distinct design of LSTM, in contrast to conventional RNNs, aids in preventing the vanishing gradient issue. The memory cell and the three different kinds of gates input, forget, and output allow it to accomplish this. These elements give LSTM great efficacy for sequence labeling tasks by enabling it to add or remove information from the cell state over extended sequences.

- **Decoding (CTC):**

A kind of loss function called CTC is applied to sequence issues in which time is uncertain. It is applied in situations where we are unsure of the alignment between the labels and the input data, which frequently occurs in speech and handwriting recognition. In order to provide a prediction with a variable length, CTC appends a blank label in addition to existing labels. It is then perfect for OCR decoding tasks

because it computes loss by adding up all possible alignments of the input to the target sequences.

An altered version of the deep-text-recognition-benchmark architecture serves as the training pipeline for EasyOCR's recognition execution. With the use of this framework, text recognition models can be trained on a variety of datasets, making EasyOCR incredibly versatile and effective.

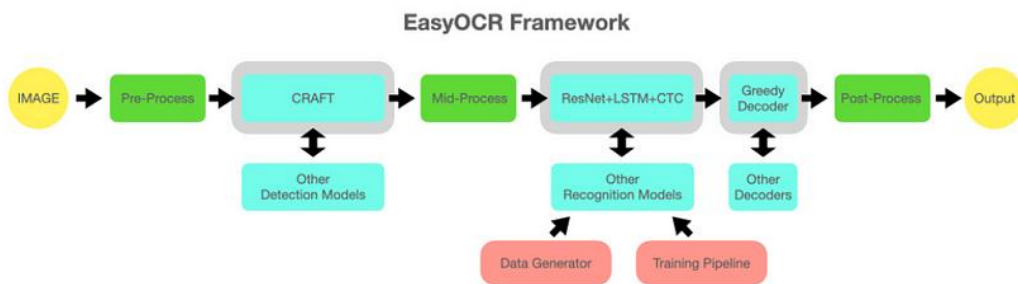


Figure 2.1 EasyOCR Framework

2.2 Text to Speech (TTS):

TTS is a library for advanced Text-to-Speech generation. It's built on the latest research, was designed to achieve the best trade-off among ease-of-training, speed and quality. TTS comes with pretrained models (e.g. Concatenative TTS, Parametric TTS, Hidden Markov Model (HMM)-based Synthesis, Neural TTS, Tacotron & WaveNet), tools for measuring dataset quality and already used in 20+ languages for products and research projects.

2.1 Google Text to Speech (gTTS):

gTTS, is a special tool created by Google. It turns written words or extracted text from images into speech, making it easier for us to hear what's written on a screen. This is super helpful for people who have trouble seeing. gTTS is known for being clear, easy to use, and it works well with other programs, which is why so many people like to use it.

It's part of the Python library, which is a collection of tools that help programmers make cool stuff. gTTS is great for reading out loud from books or for giving voice commands, making it a useful tool in our digital world.

It is like a smart robot that knows how to turn text into speech. When you give it something to read, it breaks it down into smaller parts. This step is important because it helps gTTS read the text correctly and naturally.

It uses special rules, similar to the ones in the Google Translate Text-to-Speech API, to make sure the speech sounds just like a human talking.

gTTS checks each part of the text to decide how it should sound. The final speech then comes out in a way that's easy to listen to and understand.

It can speak in many different languages and voices. It's not just for English. You can use it for French, Arabic, and lots of other languages too.

This is really helpful for people who make apps or websites for users all around the world. Users can pick different accents and voices.

This feature is especially useful in our project that it can serve several visually impaired nationalities.

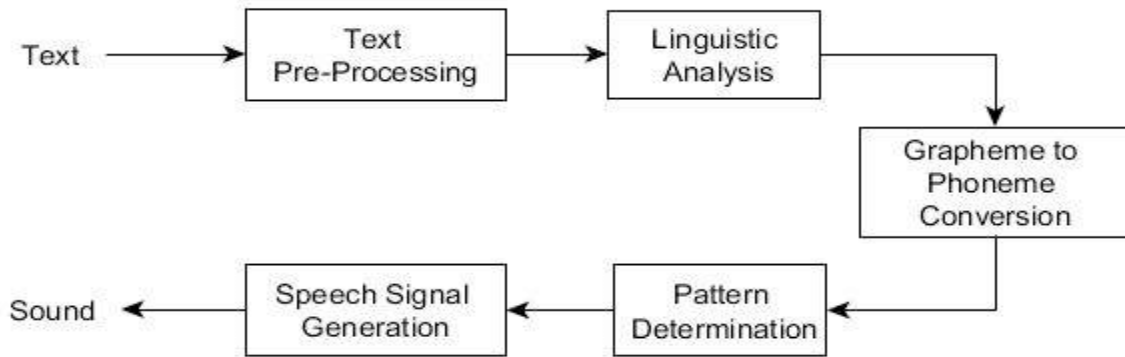


Figure 2.2 gTTS converting text to speech architecture

2.3 Yolo8 Model:

YOLOv8 represents the latest evolution in the YOLO series of real-time object detection systems, setting new standards in both accuracy and speed. It builds upon the innovations of its predecessors, integrating advanced features and optimizations that enhance its capability for detecting objects swiftly and accurately across diverse applications. This iteration of YOLO not only refines existing technologies but also introduces novel enhancements, making it highly suitable for a broad spectrum of object detection tasks with exceptional performance.

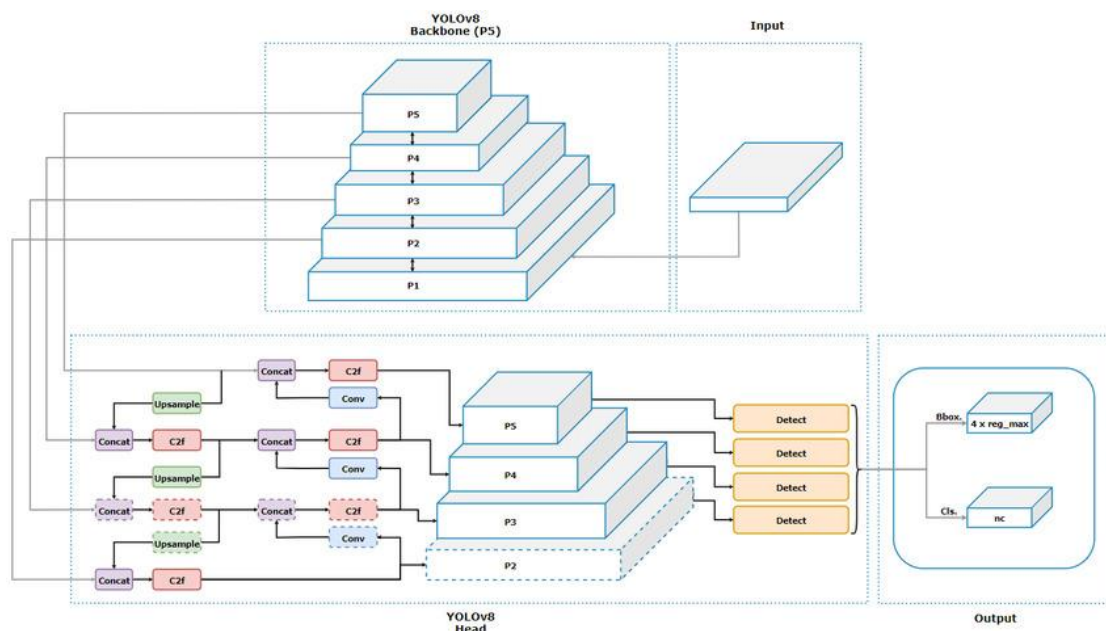


Figure 2.3 YOLOv8 model

Key Features

YOLOv8 integrates cutting-edge backbone and neck architectures, enhancing its capability for superior feature extraction and precise object detection.

Utilizing an anchor-free split Ultralytics head, YOLOv8 achieves heightened accuracy and efficiency in detecting objects, surpassing traditional anchor-based methods.

The model strikes an optimized balance between accuracy and speed, making it exceptionally well-suited for real-time object detection across diverse applications. Offering a diverse selection of pre-trained models, YOLOv8 provides flexibility to meet varying task requirements and performance metrics, ensuring suitability for a wide range of specific use cases.

The YOLOv8 series presents a comprehensive array of models, each finely tuned for distinct computer vision tasks. These models are meticulously crafted to meet diverse needs, ranging from fundamental object detection to advanced applications such as instance segmentation, pose/keypoints detection, oriented object detection, and classification.

Every iteration within the YOLOv8 series is meticulously optimized for its specific task, guaranteeing exceptional performance and precision. Furthermore, these models seamlessly support multiple operational modes including Inference, Validation, Training, and Export enabling their versatile deployment across various stages of development and deployment.

Model	Filenames	Task	Inference	Validation	Training	Export
YOLOv8	yolov8n.pt yolov8s.pt yolov8m.pt yolov8l.pt yolov8x.pt	Detection	✓	✓	✓	✓
YOLOv8-seg	yolov8n-seg.pt yolov8s-seg.pt yolov8m-seg.pt yolov8l-seg.pt yolov8x-seg.pt	Instance Segmentation	✓	✓	✓	✓
YOLOv8-pose	yolov8n-pose.pt yolov8s-pose.pt yolov8m-pose.pt yolov8l-pose.pt yolov8x-pose.pt yolov8x-pose-p6.pt	Pose/Keypoints	✓	✓	✓	✓
YOLOv8-obb	yolov8n-obb.pt yolov8s-obb.pt yolov8m-obb.pt yolov8l-obb.pt yolov8x-obb.pt	Oriented Detection	✓	✓	✓	✓
YOLOv8-cls	yolov8n-cls.pt yolov8s-cls.pt yolov8m-cls.pt yolov8l-cls.pt yolov8x-cls.pt	Classification	✓	✓	✓	✓

Figure 2.4 Overview of the YOLOv8 model variants

MS COCO: A Key Dataset for Object Detection: -

The Microsoft Common Objects in Context (MS COCO) dataset is a widely used resource in the field of computer vision, particularly for training and evaluating object detection algorithms. MS COCO provides a rich and diverse set of images, annotated with labels for multiple objects, their bounding boxes, and segmentation masks. This dataset is pivotal for developing robust object detection models that perform well in real-world scenarios.

Key Features of MS COCO:

Diverse Objects: MS COCO includes annotations for 80 object categories, ranging from everyday items like chairs and tables to animals and vehicles. This diversity ensures that detection models trained on MS COCO can recognize a wide variety of objects in different contexts.

Complex Contexts: The dataset emphasizes objects in complex scenes, providing annotations for objects that are partially occluded or in cluttered environments. This complexity is essential for developing models that can handle real-world visual challenges.

Detailed Annotations: Each image in MS COCO is annotated with object bounding boxes, segmentation masks, and keypoints for human poses. These detailed annotations allow for the training of models capable of precise object localization and recognition.

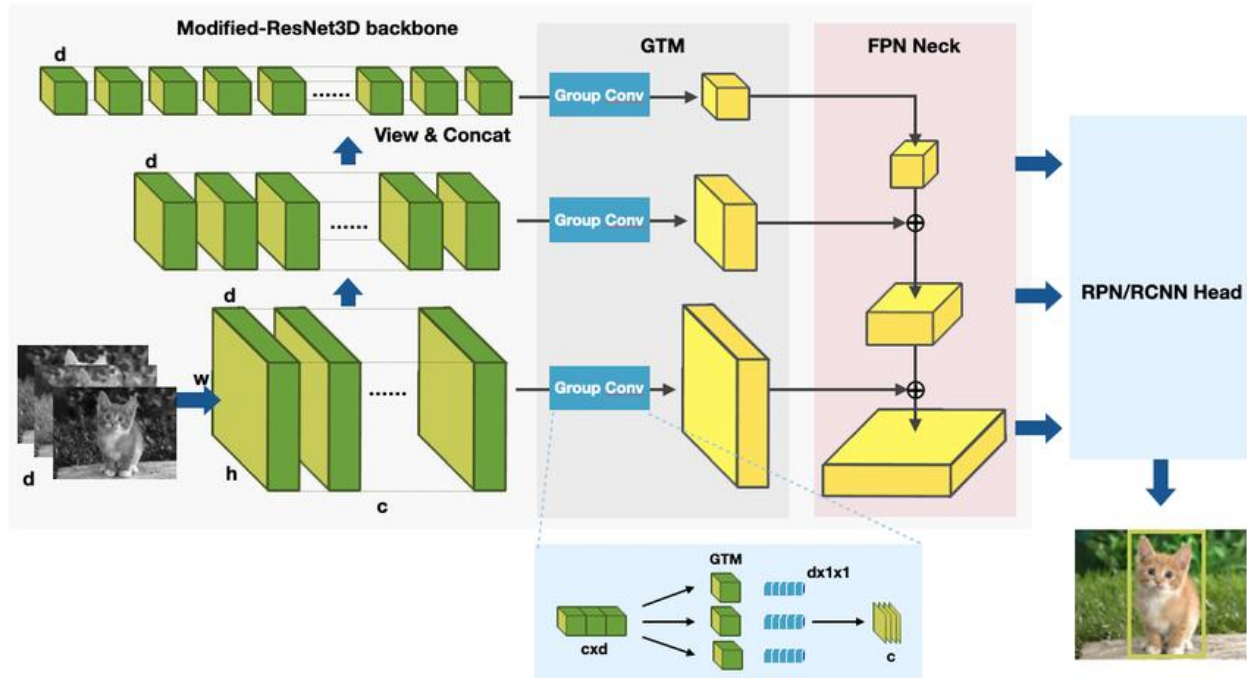


Figure 2.5 Object Detection Workflow Using MS COCO

2.4 OpenCv:

OpenCv is a huge open-source library for computer vision, machine learning, and image processing. Now, it plays a major role in real-time operation which is very important in today's systems. By using it, one can process images and videos to identify objects, faces, or even the handwriting of a human.

When it is integrated with various libraries, such as NumPy, python is capable of processing the openCv array structure for analysis. To Identify an image pattern and its various features we use vector space and perform mathematical operations on these features.

OpenCV has many important features, the main features are Cross-platform, Portable, Fast, Open source & Vast algorithms.

OpenCV is a crucial tool in computer vision, providing strong and flexible functions for image and video processing. Using OpenCV in our project made face

recognition, OCR, and object detection more effective and accurate, helping us create a smart system to assist visually impaired and blind individuals.

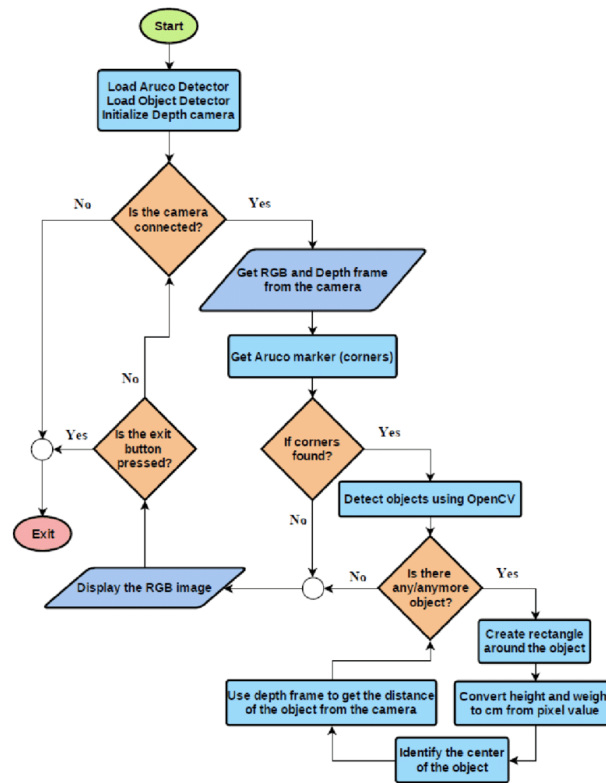


Figure 2.6 Flowchart of the proposed OpenCV algorithm

2.5 Face_Reognition Library:

The "face_recognition" library is a powerful and easy-to-use tool for face detection and recognition tasks. It wraps around the Dlib and OpenCV libraries, leveraging their robust functionalities to provide an efficient face recognition solution. This library simplifies the process of detecting and recognizing faces in images and videos, making it a valuable asset for projects that require facial analysis.

Key Algorithms in the "face_recognition" Library:

- **HOG (Histogram of Oriented Gradients):**

The HOG algorithm detects faces by analyzing the gradient structure in an image. It works by dividing the image into small cells, computing the gradient direction for each cell, and then creating a histogram of gradient directions for each cell. These histograms are combined to form a descriptor that represents the face. It is effective in detecting faces in various orientations and lighting conditions, making it a reliable method for face detection.

- **Face Embedding Model (ResNet-34):**

The face embedding model in the "face_recognition" library is based on the ResNet-34 architecture. This deep learning model processes an image of a face and outputs a 128-dimensional vector. Each vector represents the essential features of a face, allowing for accurate comparison between different faces. It ensures that the face embeddings are highly discriminative, which is crucial for recognizing and distinguishing between different individuals.

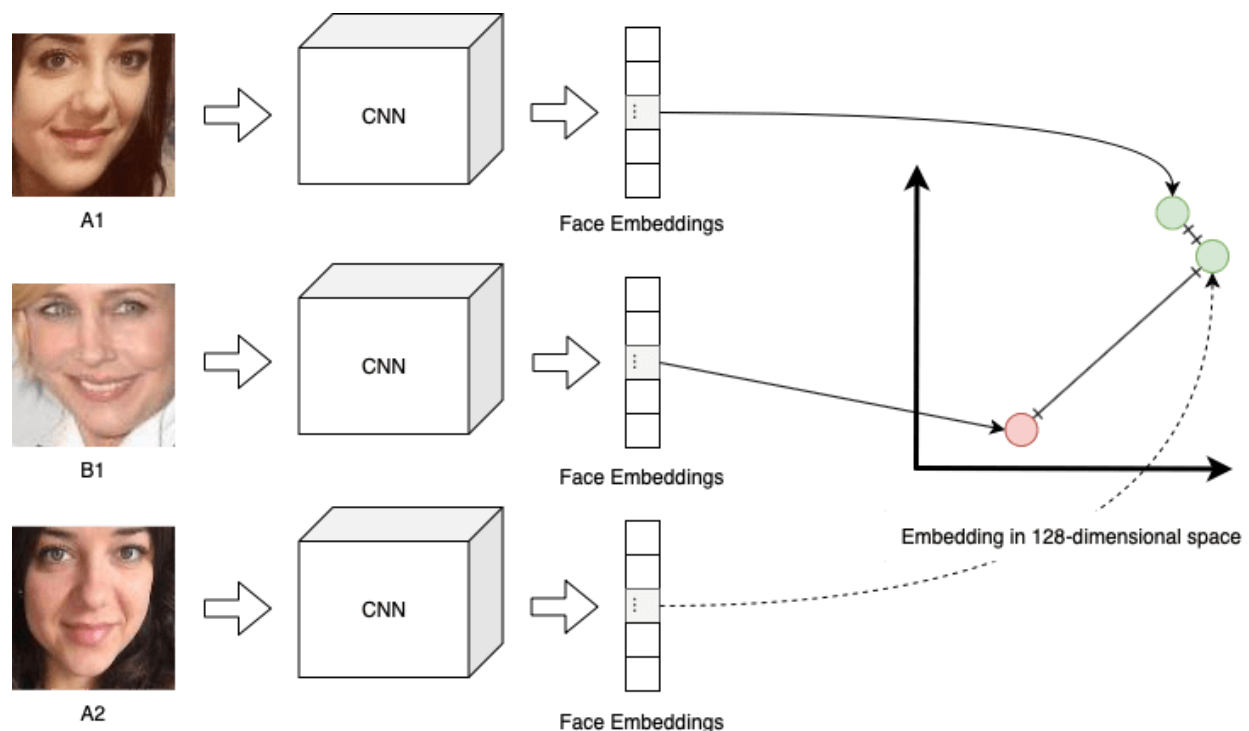


Figure 2.7 Generating and Comparing Face Embeddings in 128-Dimensional Space Using CNN

- **SVM (Support Vector Machine):**

The SVM is a machine learning classifier that operates on the 128-dimensional face embeddings generated by the ResNet-34 model. It is trained on a labeled dataset of face embeddings, learning to separate the feature vectors of different individuals. When a new face embedding is provided, the SVM can accurately classify it based on the learned boundaries.

It known for their effectiveness in classification tasks, particularly when dealing with high-dimensional data like face embeddings. They help in achieving high accuracy in face recognition.

2.6 Literature Review:

This section gives an overview of each part of our project. Each part had its own challenges, so we tackled them one by one. We found solutions by reading different papers and studies that matched each part's problems. This section sets the stage for the next parts, where we'll explain in detail the experiments we did for each part and what we learned.

2.6.1 Object Detection:

Object detection is a pivotal technology in assisting individuals who are visually impaired, enabling them to identify and locate objects within images or video frames. This functionality significantly enhances accessibility and independence. By integrating Text-to-Speech (TTS) capabilities, these systems can convert detected object information into spoken feedback, providing real-time assistance to users.

Object Recognition System for the Visually Impaired: A Deep Learning Approach using Arabic Annotation: -

This introduces a method for detecting objects in images to assist Arabic-speaking individuals with visual impairments. Utilizing the Mask R-CNN algorithm, the system accurately locates objects and provides their names and positions in Arabic via a TTS system. The model achieved an impressive accuracy of 83.9% when evaluated with standard datasets. This research addresses a critical gap by focusing on Arabic language support, which is often limited in existing object detection studies. The integration of Arabic annotations and TTS capabilities ensures that visually impaired users receive precise and culturally relevant assistance, enhancing their interaction with their environment ^[5].

Object Detection and Conversion of Text to Speech for Visually Impaired: -

This describes a two-stage system designed to assist visually impaired individuals by first capturing an image using a portable camera module and then identifying objects such as cell phones, people, or books by matching them with a predefined dataset. In the second stage, the system synthesizes recognized text into speech using the gTTs module and iPython audio library for TTS conversion. The goal is to provide natural language audio feedback, aiding not only visually impaired individuals but also anyone needing rapid text-to-speech conversion. The system, implemented in Python, leverages the YOLOv4 model trained on the MS COCO dataset, ensuring high accuracy and efficiency in object detection ^[6].

2.6.2 Face Recognition:

Face recognition is important for enhancing the independence and security of visually impaired individuals. This technology enables the identification and verification of familiar faces within images or video streams, making interactions easier and improving social connectivity. Integrating Text-to-Speech (TTS) capabilities allows systems to audibly announce the presence of recognized relatives.

Real-Time Face Recognition System Using KPCA, LBP and Support Vector Machine ^[7], This paper presents a real-time face recognition system using advanced techniques like Kernel Principal Component Analysis (K-PCA), Local Binary Pattern (LBP), and Support Vector Machine (SVM). It aims to enhance biometric systems given rising security concerns and technological progress. Face recognition is challenging due to variations in lighting and facial expressions. The study demonstrates that K-PCA combined with LBP and SVM performs better than traditional methods like PCA in accurately recognizing faces.

Face Recognition Assistant for People with Visual Impairments ^[8], The system developed in the paper uses advanced face recognition technology to assist visually impaired individuals in recognizing people they interact with. It utilizes the MTCNN algorithm for face detection and estimates head poses based on predefined distance and orientation parameters. The system automatically captures high-quality face samples during interactions, filtering out images that are blurry or

poorly illuminated. For recognition, it employs neural networks to extract distinctive features from faces and utilizes a K-nearest neighbor classifier with cosine similarity metrics to identify individuals. This approach aims to provide accurate and reliable assistance to visually impaired users in social interactions.

2.6.3 Text Recognition:

Text recognition technology enables the identification and extraction of text from images or video streams, facilitating tasks such as reading signs, labels, and documents. Integrating Text-to-Speech (TTS) capabilities allows systems to audibly announce the recognized text, providing real-time assistance in navigating environments and accessing information.

An Assistive Reading System For Visually Impaired Using OCR And TTS ^[9], This paper introduces a system designed to help people who are blind or visually impaired. It reads text from scanned documents and turns it into spoken words using special technology. The system uses Optical Character Recognition (OCR) to find text on pages and Text-to-Speech (TTS) to change that text into natural-sounding speech. It works well even with different fonts, colors, and backgrounds on documents.

Text to Speech Conversion ^[10], This paper presents a system using Raspberry Pi that combines OCR and TTS technologies to help visually impaired users interact with text images through spoken output. It consists of two modules: one captures and processes images to extract text using OCR, while the other converts this text into audible speech. The device operates independently, is portable, and supports recognition of various text formats and numbers, enhancing accessibility for users without requiring an internet connection.

Chapter 3

Analysis and Design

3.1 System Overview

3.1.1 System Architecture

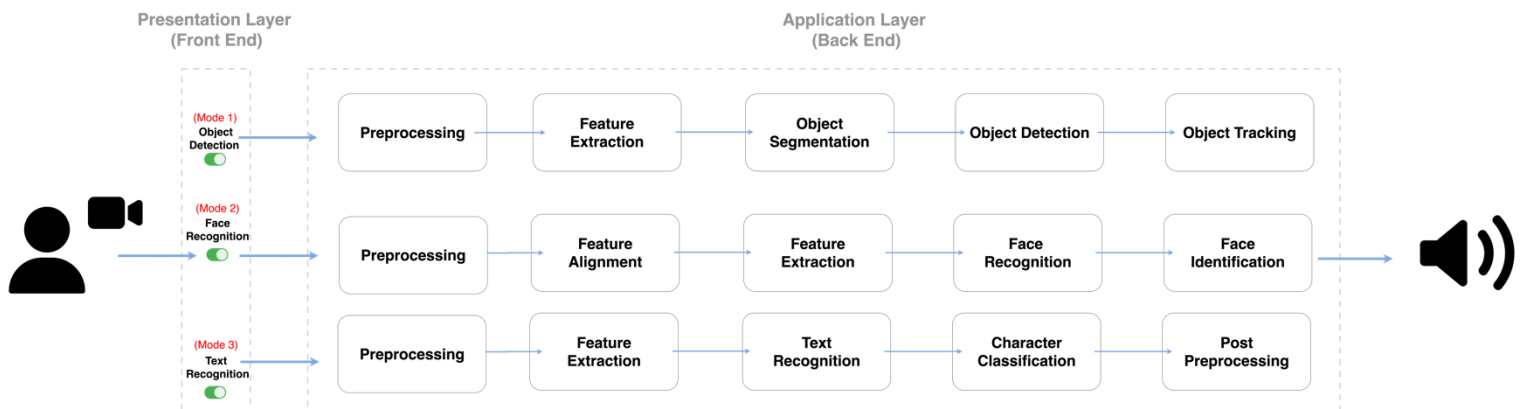


Figure 3.1 System Architecture Diagram

- **Presentation Layer:**

responsible for interacting with the user and handling the initial input. It provides the interface through which users can select the mode of operation (Object Detection, Face Recognition, or Text Recognition). This layer captures the video

feed from the user's camera or video source, preparing it for further processing by the Application Layer. Based on the user's mode selection, the system routes the video feed to the appropriate processing pipeline in the backend. Overall, the Presentation Layer acts as the bridge between the user and the system.

- **Application Layer:**

responsible for processing the video feed according to the selected mode of operation. Initially, it performs preprocessing to enhance the quality or format of the video feed. Next, it extracts relevant features from the preprocessed frames, tailored to the chosen mode—whether object features, facial features, or text features.

3.1.2 Functional Requirements

1. Face Recognition:

- The system is equipped to recognize and identify faces using the “face_recognition” library, a popular tool for face detection and recognition.
- It provides audible feedback to users upon identifying individuals, enhancing interaction through spoken notifications.
- The system includes functionality to store and manage profiles of known individuals, facilitating personalized interactions and user-specific information.

2. Object Detection:

- The system is designed to detect and identify objects in real-time using its camera module.
- It utilizes the YOLO model, known for its high accuracy and efficiency in object detection tasks.
- Upon detecting objects, the system provides audible feedback to inform the user about the presence and identity of detected objects.

3. Optical Character Recognition (OCR):

- The system utilizes EasyOCR technology for text extraction from images or live video feeds.
- convert printed or handwritten text into digital format.

4. Text-to-Speech (TTS):

- Implements TTS technology to convert recognized text into spoken words.
- Provides auditory feedback of the extracted text to the user.

5. Mode Selection:

- Allows users to switch between different modes (Face Recognition, Object Detection, Text Recognition) via a dedicated toggle button in the application.
- Each mode utilizes TTS to provide audio feedback relevant to the selected mode.

3.1.3 Nonfunctional Requirements

1. Performance:

- The system should provide real-time processing for object detection, face recognition, and text-to-speech conversion with minimal latency.
- TTS conversion shall produce clear and natural-sounding speech with high accuracy to ensure effective communication.

2. Reliability:

- The system should maintain a high level of accuracy and reliability in object detection, face recognition, and TTS operations.
- It should operate robustly under various environmental conditions without frequent crashes or errors.

3. Usability:

- The user interface should be intuitive and accessible to individuals with visual impairments, promoting ease of use and independence.
- Voice prompts and feedback should be clear, understandable, and contextually relevant to enhance user interaction.

4. Scalability:

- The system architecture should support future enhancements and additional features, allowing for seamless integration of new functionalities.
- It should accommodate upgrades in hardware, sensors, and software modules to meet evolving user needs and technological advancements.

5. Security:

- The system should prioritize user data privacy, particularly for facial profiles and personal information.
- Secure communication protocols should be used for data transmission to prevent unauthorized access and ensure confidentiality.

6. Compatibility:

- The system should be compatible with a variety of mobile devices and operating systems, enhancing flexibility and usability across different platforms.
- Hardware components should be interoperable, allowing for seamless integration and efficient performance across the system.

7. Battery Life and Portability:

- The wearable device should be energy-efficient to maximize battery life, ensuring prolonged usage without frequent recharging.
- It should be lightweight, comfortable to wear, and portable, promoting user comfort and encouraging continuous use throughout the day.

3.1.4 System Users

A. Intended Users:

1. Primary Users

- Individuals with visual impairments who need assistance in navigating and interacting with their environment.
- Elderly individuals with declining vision who require help in recognizing objects and people.

2. Secondary Users

- Caregivers and family members of visually impaired individuals who use the system to assist their loved ones.
- Healthcare professionals and organizations provide support to visually impaired individuals.

B. User Characteristics

1. Visual Impairment

- Users may have varying degrees of visual impairment, from partial to complete blindness.
- Users rely on auditory feedback and tactile interactions for navigation and information access.

2. Age Range

- Users may range from children to elderly individuals, necessitating an intuitive and adaptable interface.

3. Technical Proficiency

- Users may have different levels of technical proficiency, from tech-savvy individuals to those with limited experience with technology.
- The system should be easy to use regardless of the user's technical background.

4. Mobility

- Users may have varying levels of mobility, with some relying on additional mobility aids such as canes or guide dogs.
- The system should support hands-free operation to accommodate users with limited mobility.

5. Cognitive Abilities

- Users may have different cognitive abilities, requiring the system to provide simple and clear instructions.
- The interface should avoid complex interactions and provide straightforward navigation.

3.2 System Analysis & Design

3.2.1 Use Case Diagram

- Users must verify their password during login.
- Login display errors may not be shown explicitly.

- Adding relatives during the signup process is necessary to utilize the calling feature.

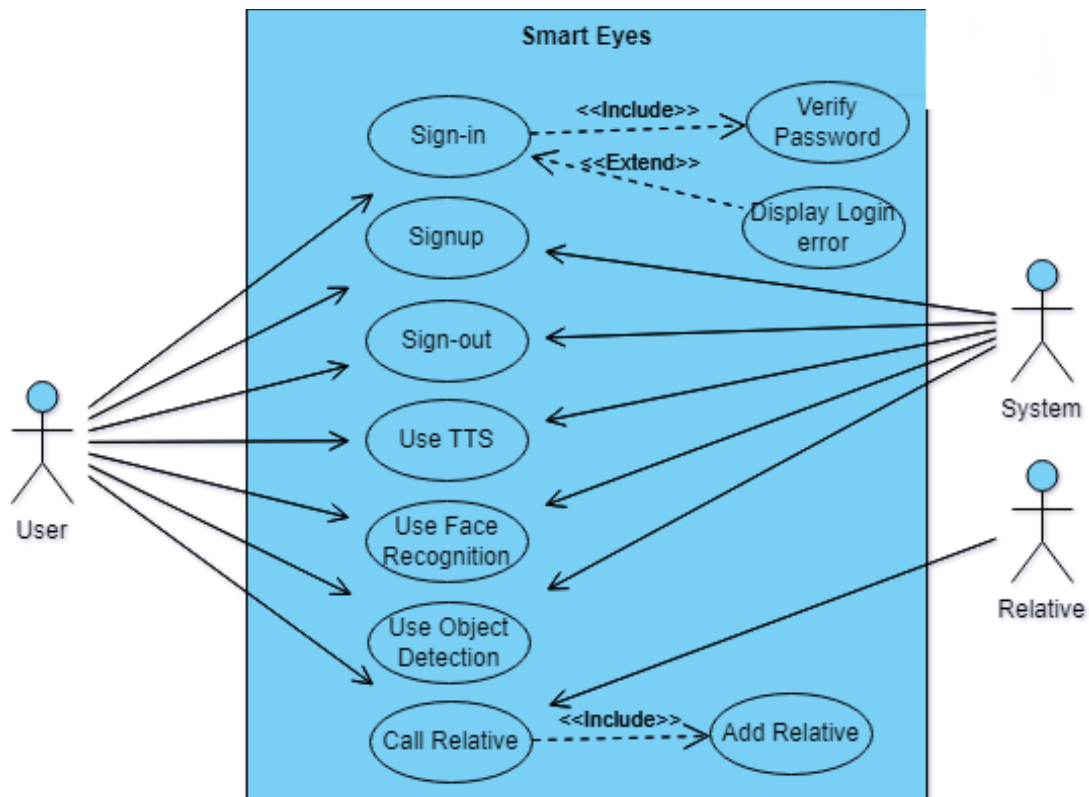


Figure 3.2 Use Case Diagram

3.2.2 Class Diagram



Figure 3.3 Class Diagram

This class diagram comprises three primary classes: User, System Administrator, and Relative. Each class encapsulates specific attributes and methods that define its role and functionality within the system.

- The User class is central to the diagram, representing the end-users who interact with the system. Users can perform essential actions such as signing up, signing in, and signing out. They can also utilize advanced features like calling a relative, using text-to-speech (TTS), face recognition, and object detection.
- The System Administrator class handles the administrative tasks, such as signing in, approving user sign-ups and sign-ins, and signing out. This ensures that the system operates smoothly and securely, with administrators overseeing user activities and maintaining control over access and permissions.

- The Relative class connects to the User class, allowing users to store and manage the contact details of their relatives. This relationship is depicted as a one-to-many connection, meaning each user can have multiple relatives, but each relative is associated with only one user.

3.2.3 Sequence Diagrams

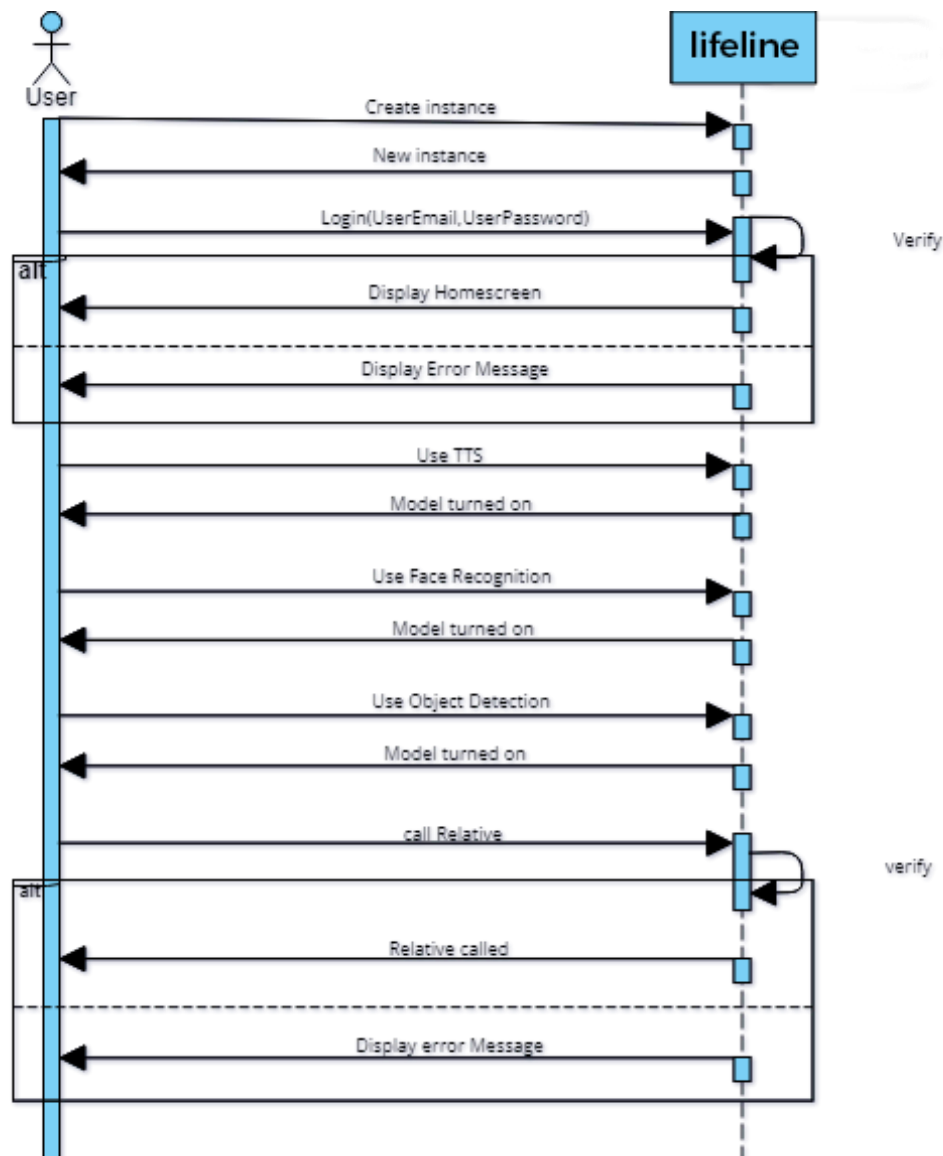


Figure 3.4 Sequence Diagram

Chapter 4

Implementation

4.1 Functions used in OCR & TTS:

Function `capture_frame()`:

This Function is used to capture a single frame from the live video feed. It Returns a frame (A BGR image frame if the capture is successful) or None (If the capture fails).

Steps:

- **Initialize Video Capture:** A video capture object is created using OpenCV's `'cv2.VideoCapture(0)'` to access the default camera.
- **Capture Frames Continuously:** The script enters a loop to continuously capture frames from the live video feed.
- **Display Frames:** Each captured frame is converted from BGR to RGB format and displayed using Matplotlib.
- **User Input Handling:**
 - If the user presses a current button, the current frame is saved as an image file, processed with OCR and text-to-speech(in `Ocr_Tts` function), and then deleted.
 - If the user presses another current button, the loop breaks, and the video capture is released.

Function Ocr_Tts(frame):

Steps:

- Read the Image: The image is read using OpenCV's `cv2.imread`.
- Initialize OCR Reader: An EasyOCR reader is initialized for the English language.
- Text Detection: The OCR reader detects text in the image.
- Text Recognition: For each detected text segment, if the confidence score is above a threshold, the text is added to the recognized text string.
- Image Annotation: The detected text is annotated on the image with bounding boxes and text labels.
- Display Image: The annotated image is displayed using Matplotlib.
- Text-to-Speech Conversion: The recognized text is converted to speech using gTTS and played.

Techniques and algorithms implemented:

1. Live Video Capture with OpenCV

Capture live video frames from the default camera.

`cv2.VideoCapture(0)` is used to initialize video capture. The `capture_frame` function captures individual frames using `cap.read()`.

2. Real-time Frame Display with Matplotlib:

Display captured video frames in real-time. Captured frames are converted from BGR to RGB using `cv2.cvtColor` and displayed using Matplotlib's `plt.imshow`, `plt.pause`, and `plt.draw` methods to continuously update the displayed frame.

3. User Interaction for Frame Capture:

Capture and process a frame based on user input. The script waits for user input to either save and process the current frame or quit the loop depending on a pressed button. The `cv2.imwrite` method saves the frame, and the `Ocr_Tts` function processes the saved image for OCR and text-to-speech.

4. Optical Character Recognition (OCR) with EasyOCR:

Recognize and extract text from images. EasyOCR is used to detect and recognize text in the captured frame. Detected text is filtered based on a confidence threshold and annotated on the image.

5. Text-to-Speech (TTS) with gTTS:

Convert recognized text to audible speech. The recognized text is converted to speech using Google's Text-to-Speech (gTTS) API, and the resulting audio is played back to the user.

Technologies Used in Implementation:

1. Matplotlib:

It is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is used in our project to display live video frames captured from the camera in real-time, providing a visual interface for the user to interact with.

2. OpenCV:

OpenCV (Open Source Computer Vision Library) is a popular library that provides tools for real-time computer vision. It is used in our project for capturing video frames from the camera, converting color formats, and saving frames as image files.

3. EasyOCR:

It is used in our project to detect and recognize text in captured frames, which is then used to assist visually impaired users.

4. gTTS:

It is used in our project to convert the recognized text into speech, which is then played back to the user, providing an auditory representation of the text.

4.2 Functions used in Face Recognition:

Function ``faces_encode(images)``:

This function encodes known face images into feature vectors using the ``face_recognition`` library. It converts each image from BGR to RGB format using ``cv2.cvtColor``. Then uses the ``face_recognition`` library to encode the faces in the images, producing 128-dimensional feature vectors for each face. It stores the encoded vectors in a list and returns the list.

Main Loop for Face Recognition:

This is the main loop where face recognition is performed on each captured frame.

Steps:

- **Initialize Video Capture:** Create a video capture object using OpenCV's ``cv2.VideoCapture(0)`` to access the default camera.
- **Capture Frames Continuously:** Enter a loop to continuously capture frames from the live video feed.
- **Resize and Convert Frame:** Resize each captured frame to 25% of its original size and convert it from BGR to RGB format.
- **Face Detection:** Use the ``face_recognition`` library to detect face locations in the resized frame.
- **Encode Faces in Frame:** Encode detected faces into feature vectors.
- **Match Faces:** Compare the encoded faces from the frame with known encoded faces to find matches, using a tolerance of 0.5.
- **Determine Best Match:** Consider the face with the smallest distance as the best match.
- **Draw Bounding Boxes:** Draw bounding boxes around detected faces and display names.
- **Text-to-Speech:** Convert recognized text to speech using gTTS and play it if a match is found, ensuring that each person's name is announced only once.

Techniques and Algorithms Implemented:

1. Face Encoding with face_recognition:

It encodes known face images into 128-dimensional feature vectors using deep learning models based on the ResNet-34 architecture.

The used algorithm: 'face_recognition.face_encodings'.

2. Face Detection and Recognition:

It detects and recognizes faces in real-time video frames.

The Algorithm 'face_recognition.face_locations' for detecting face locations and 'face_recognition.compare_faces' for matching faces.

3. Real-time Video Capture and Processing:

It captures and processes video frames in real-time.

OpenCV's 'cv2.VideoCapture' for video capture, 'cv2.cvtColor' for color conversion, and 'cv2.resize' for resizing frames.

4. Text-to-Speech Conversion with gTTS:

It converts recognized text to speech using gTTS.

Technologies Used in Implementation:

1. OpenCV:

Used for capturing video frames, converting color formats, resizing images, and drawing bounding boxes and text on frames.

2. face_recognition Library:

Used for encoding known faces, detecting and recognizing faces in frames, and comparing face encodings.

3. gTTS:

Used to convert recognized text into speech, providing auditory feedback.

4.3 Functions used in Object Detection:

Main Loop for Object Detection:

This loop captures frames from the live video feed, processes them using a YOLO model to detect objects, and announces detected objects using text-to-speech.

Steps:

- **Initialize Video Capture:** Create a video capture object using OpenCV's `cv2.VideoCapture(0)` to access the default camera.
- **Check Camera:** Ensure the camera is opened successfully.
- **Initialize Model:** Load the YOLO model for object detection.
- **Capture Frames Continuously:** Enter a loop to continuously capture frames from the live video feed.
- **Object Detection:** Use the YOLO model to predict objects in the captured frame.
- **Extract Object Names:** Extract the names of detected objects.
- **Convert Object Names to Speech:** Use the `soundd` function to convert the detected object names into speech.
- **Exit Condition:** Break the loop if the a certain button key is pressed.

Function `soundd(objects)`:

This function uses Google Text-to-Speech (gTTS) to convert a list of detected object names into speech, saving each object's name as an audio file and playing it with a short delay between each.

Steps:

- **Iterate Over Objects:** Loop through the list of detected objects.
- **Create gTTS Object:** For each object, create a gTTS object to convert the object's name into speech.
- **Save Audio File:** Save the speech as a temporary MP3 audio file.
- **Play Audio File:** Play the audio file using the system's default media player.
- **Pause Execution:** Pause for a specified duration before proceeding to the next object.

Techniques and Algorithms Implemented:

1. Text-to-Speech Conversion:

Uses Google's Text-to-Speech API (`gTTS`) to convert text to speech.

2. Audio Playback:

Plays the generated audio files using the system's default media player.

3. Live Video Capture:

Uses OpenCV to capture live video frames.

4. Object Detection with YOLO:

Uses a pre-trained YOLO model (`yolov8l.pt`) to detect objects in the frames.

Technologies Used in Implementation:

1. gTTS:

Google Text-to-Speech library to convert text to speech.

2. Time:

Python's time module to implement a delay between audio playbacks.

3. OpenCV:

Used for capturing video frames and handling video streams.

4. YOLO (Ultralytics):

Used for object detection in real-time video frames.

4.4 Hardware

The hardware components are integrated into a pair of glasses, making it easy and convenient for the user to wear. The goal is to enhance the independence and safety of visually impaired users by providing them with critical information about their surroundings.

4.4.1 Components Used:

1. ESP32-CAM Development Board:

The ESP32-CAM is a low-cost development board that integrates a camera and Wi-Fi capabilities. It is chosen for its ability to stream video over a network and its compact size, making it ideal for wearable applications.



Figure 4.1 ESP32-CAM Development Board

2. FTDI Converter (USB to TTL)

The FTDI converter is used to program the ESP32-CAM by converting USB signals to TTL (transistor-transistor logic) signals. It facilitates the communication between the ESP32-CAM and the computer during the initial programming phase.

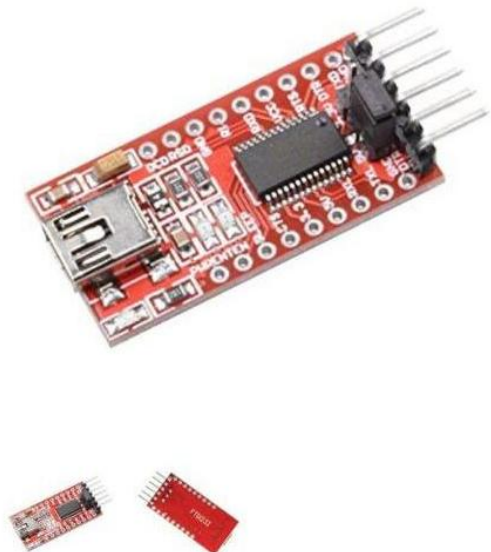


Figure 4.2 FTDI Converter (USB to TTL)

3. TP4056 Lithium Battery Charger

The TP4056 module is a linear charger for single-cell lithium-ion batteries. It provides overcharge protection and includes indicators to show charging status. It is chosen for its reliability and safety features.

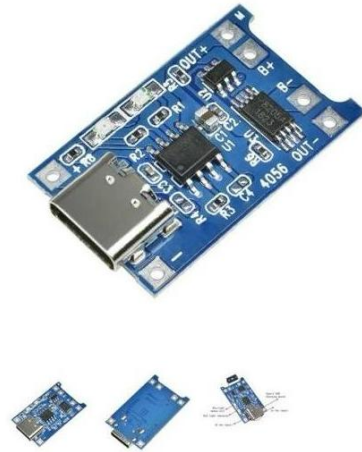


Figure 4.3 TP4056 Lithium Battery Charger

4. 18650 Lithium Ion 3.7V Battery

The 18650 lithium-ion battery is a rechargeable battery known for its high energy density, long cycle life, and reliability. Its lightweight nature (approximately 45 grams) makes it suitable for wearable devices.



Figure 4.4 18650 Lithium Ion 3.7V Battery

5. XL6009 Step-up Power Converter

The XL6009 is a DC-DC adjustable step-up (boost) converter used to increase the voltage from 3.7V (from the battery) to 5V, which is required to power the ESP32-CAM. It is chosen for its efficiency and ability to provide a stable 5V output.



Figure 4.5 XL6009 Step-up Power Converter

Component Selection

The components were carefully chosen to ensure reliability, efficiency, and lightweight design suitable for integration into wearable glasses:

1. **ESP32-CAM:** Chosen for its compact size, Wi-Fi capability, and ability to stream video.
2. **18650 Battery:** Selected for its high energy density, long cycle life, and lightweight nature (approximately 45 grams).
3. **XL6009 Step-up Converter:** Provides efficient voltage conversion from 3.7V to 5V, ensuring stable operation of the ESP32-CAM.
4. **TP4056 Charger:** Ensures safe charging and discharging of the 18650 battery with status indicators.

By using these components, the Smart Eyes project achieves a reliable and lightweight solution for assisting visually impaired individuals, providing them with real-time video streaming of their surroundings. The device is designed to be integrated into wearable glasses, making it easy and convenient to use.

4.4.2 How It's Used:

- Programming the ESP32-CAM

Steps:

1- Connect the FTDI Converter to the ESP32-CAM:

- FTDI RX to ESP32-CAM UOT
- FTDI TX to ESP32-CAM UOR
- FTDI VCC to ESP32-CAM 5V
- FTDI GND to ESP32-CAM GND

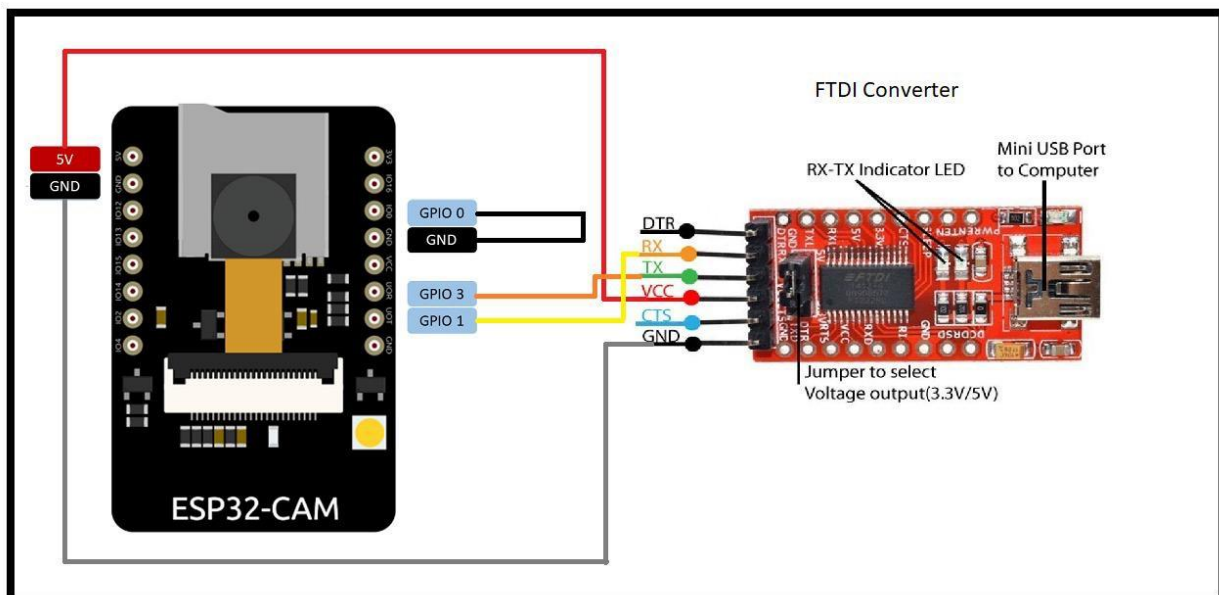


Figure 4.6 The Connection of FTDI Converter to the ESP32-CAM

2- Enter Flash Mode:

Connect IO0 to GND on the ESP32-CAM to enter flashing mode.

3- Flash the Code:

Use the Espressif development environment to upload the code to the ESP32-CAM. Espressif is the company that developed the ESP32 chipset, and they provide tools and resources for programming and developing applications for the ESP32.

4- Exit Flash Mode:

After the code is successfully flashed, disconnect IO0 from GND to allow the ESP32-CAM to run the code.

5- Power Considerations:

The ESP32-CAM is powered using the 5V pin instead of the 3.3V pin to ensure efficient operation and stable performance.

- Powering the ESP32-CAM

The ESP32-CAM is powered by a 3.7V 18650 lithium-ion battery. To achieve the required 5V for the ESP32-CAM, the following setup is used:

1- Battery to Charger:

Connect the 18650 battery to the TP4056 lithium battery charger module. The TP4056 module will manage the charging and discharging of the battery, ensuring safe operation.

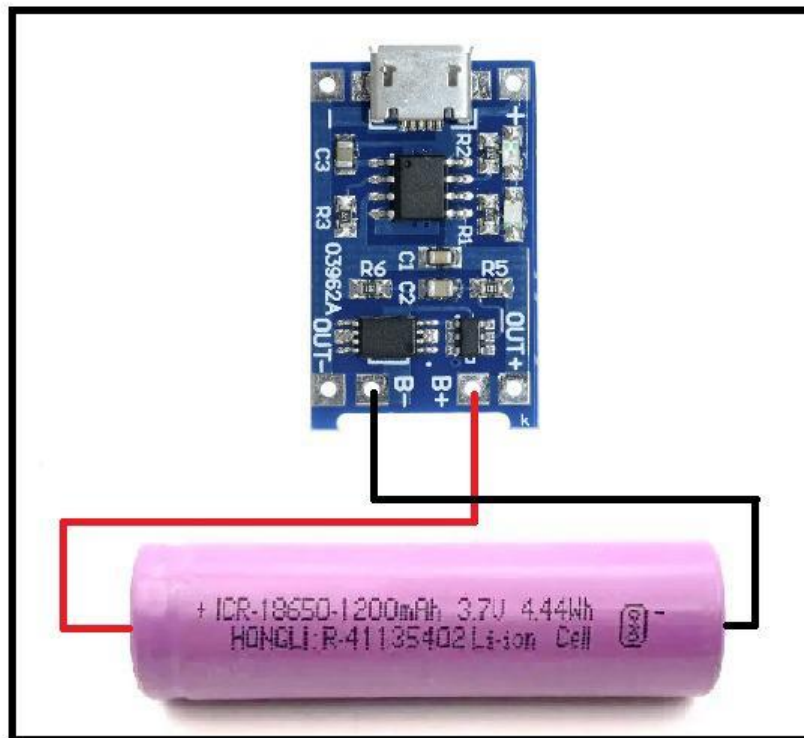


Figure 4.7 The Connection of Battery to Charger

2. Charger to Step-up Converter:

Connect the output of the TP4056 charger to the input of the XL6009 step-up converter.

3. Adjust Output Voltage:

Adjust the XL6009 step-up converter to output a stable 5V from the 3.7V input. Use a multimeter to measure the output voltage and adjust the potentiometer on the XL6009 until the output is 5V.

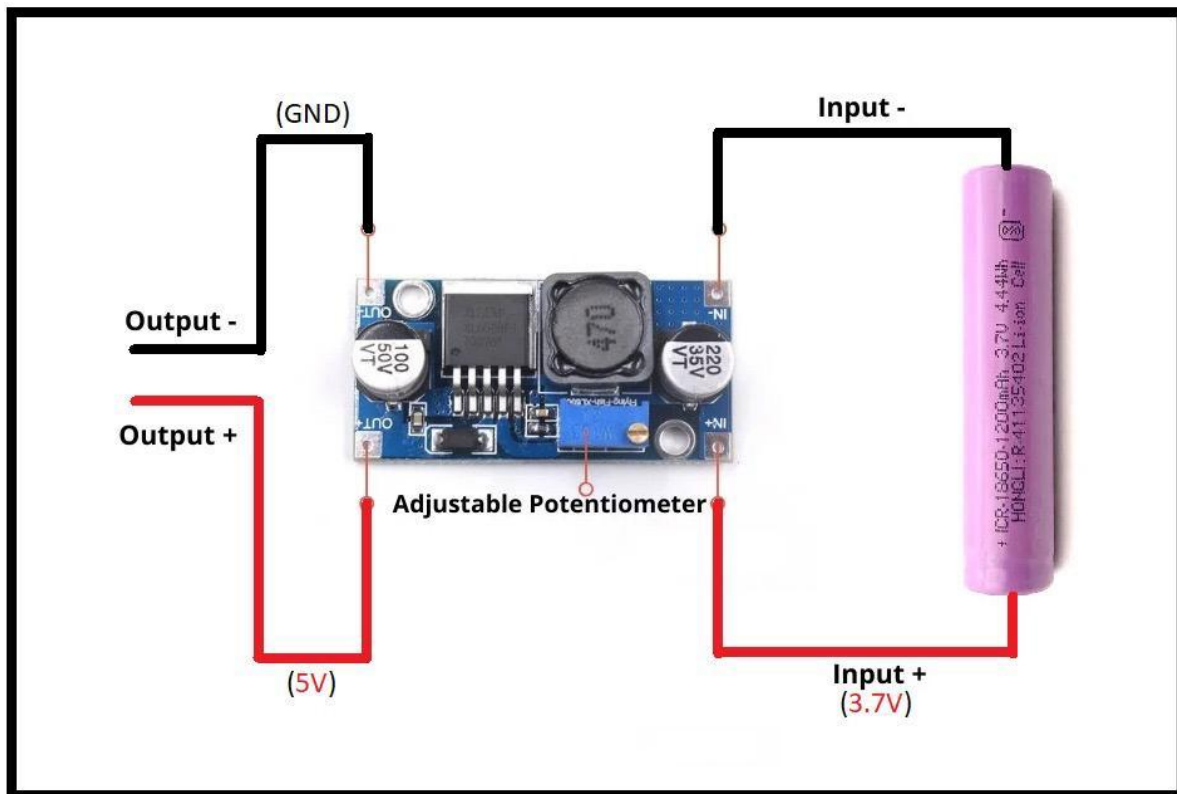


Figure 4.8 Adjust Output Voltage

4. Step-up Converter to ESP32-CAM:

Connect the 5V output of the XL6009 to the 5V pin on the ESP32-CAM. Connect the ground (GND) from the XL6009 to the GND pin on the ESP32-CAM.

- Charging the Battery

The TP4056 module charges the 18650 lithium-ion battery and provides status indicators:

- **Red LED:** Indicates the battery is charging.
- **Blue LED:** Indicates the battery is fully charged.

This module ensures that the battery is charged safely and provides overcharge protection.

4.4.3 Final Product

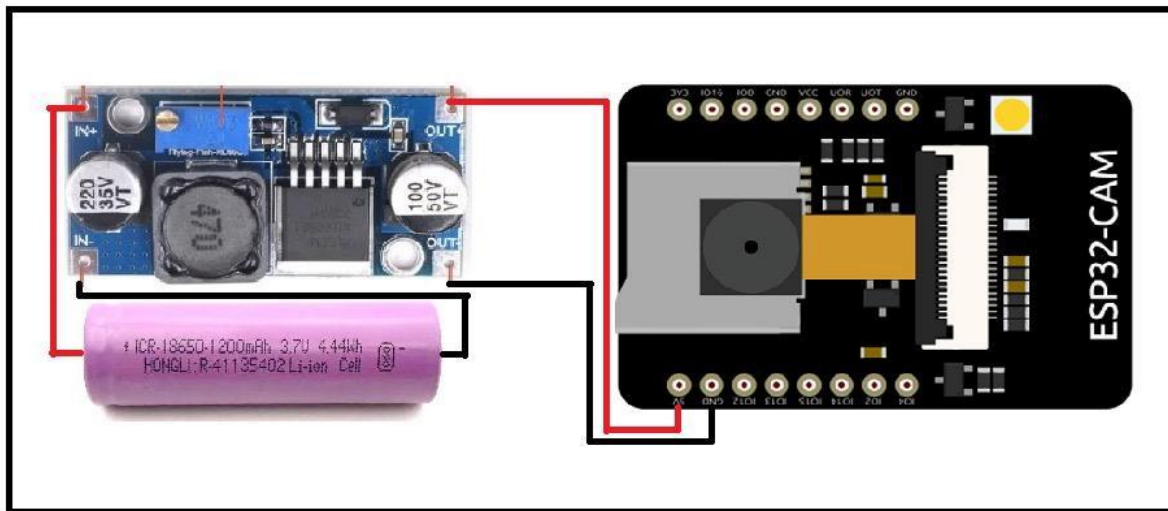


Figure 4.9 The Final Circuit of the Hardware Module

The final circuit consists of:

1. **ESP32-CAM** connected to the 5V output of the **XL6009 step-up converter**.
2. **XL6009 step-up converter** connected to the **18650 lithium-ion battery**.
3. **18650 lithium-ion battery** connected to the **TP4056 charger**.

Network Connection

The ESP32-CAM connects to a Wi-Fi network using the SSID and password provided in the flashed code. To connect:

1. Initial Setup:

When the ESP32-CAM is powered up for the first time, it connects to the specified Wi-Fi network.

The IP address assigned to the ESP32-CAM by the network is displayed on the serial monitor.

2. Accessing the Live Stream:

After pressing the RST button on the ESP32-CAM, it reconnects to the network and starts streaming video to the specified IP address.

This IP address can be accessed from any device on the same local network to view the live video stream.

Chapter 5

User Manual

1. Welcome Screen

When you first open the Smart Eyes app, you will be greeted with the welcome screen. This screen provides two options for accessing the app's features:

Login: If you already have an account, tap on the "Login" button to enter your credentials and access the app. The app will read aloud the login option.

Sign Up: If you are a new user, tap on the "Sign Up" button to create a new account. The app will read aloud the sign-up option.

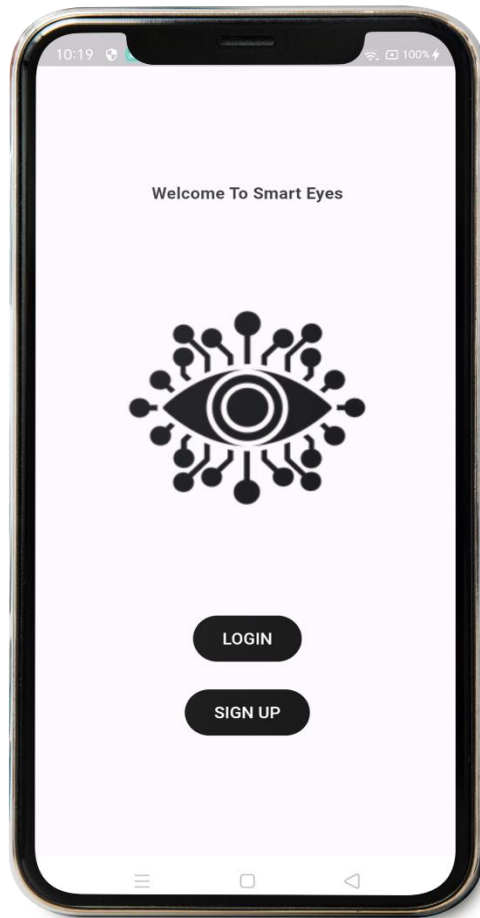


Figure 5.1 Application's Welcome Screen

2. Login Screen

If you choose to login, you will be directed to the Login screen where you need to enter your password and press the "Login" button.

At the bottom of the screen, there is a "Don't have an account? Sign Up" link. Tap on "Sign Up" to create a new account.

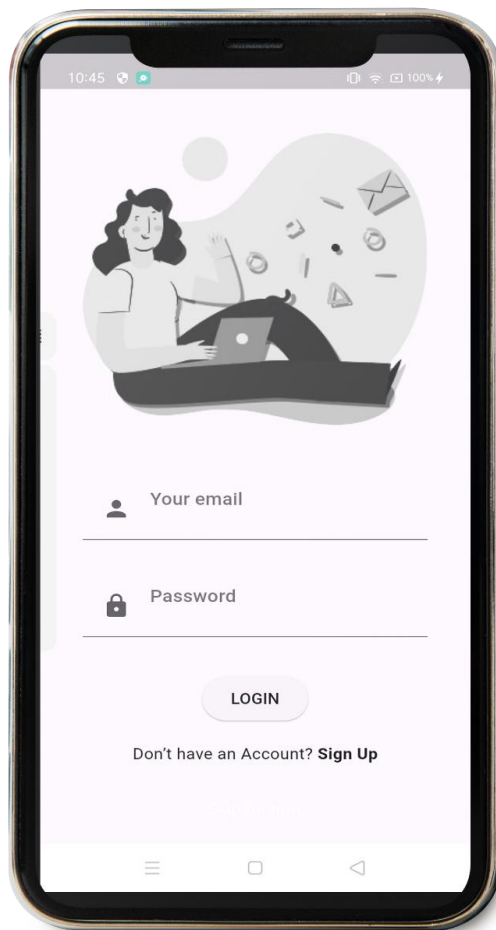


Figure 5.2 Application's Login Screen

3. Sign Up Screen

If you choose to sign up, you will be directed to the Register Account screen. Here, you need to provide the following information: email, password and re-enter the password. Also, there is an option to skip for now.

The app will prompt you to enter this information and read aloud the entered text for confirmation.

Once you have filled in your details, press the "Continue" button to complete the sign-up process. The app will read aloud the "Continue" button prompt.

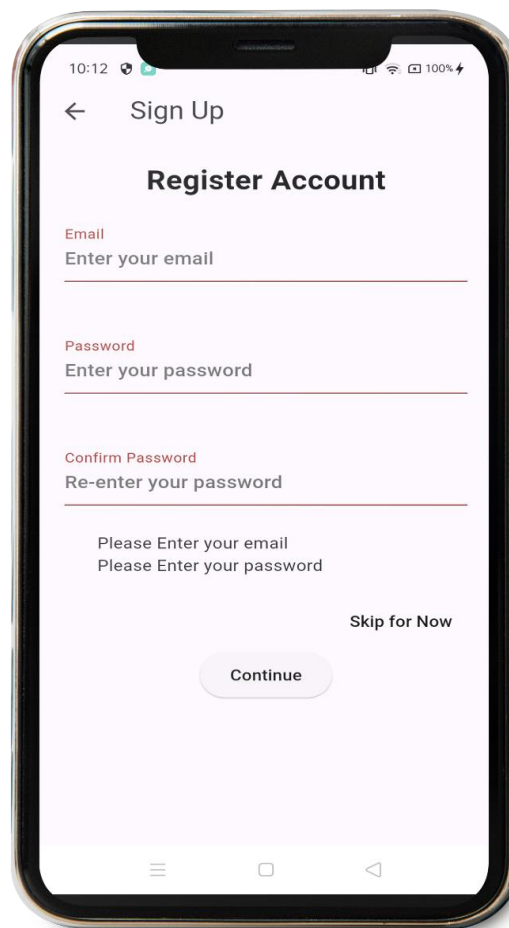


Figure 5.3 Application's Signup Screen

4. Complete Profile Screen

Then you will be directed to the Complete Profile screen. Here, you need to provide additional information to complete your profile: first name, last name, phone number and address.

The app will prompt you to enter this information and read aloud the entered text for confirmation.

Once you have filled in your details, press the "Continue" button to complete the sign-up process. The app will read aloud the "Continue" button prompt.

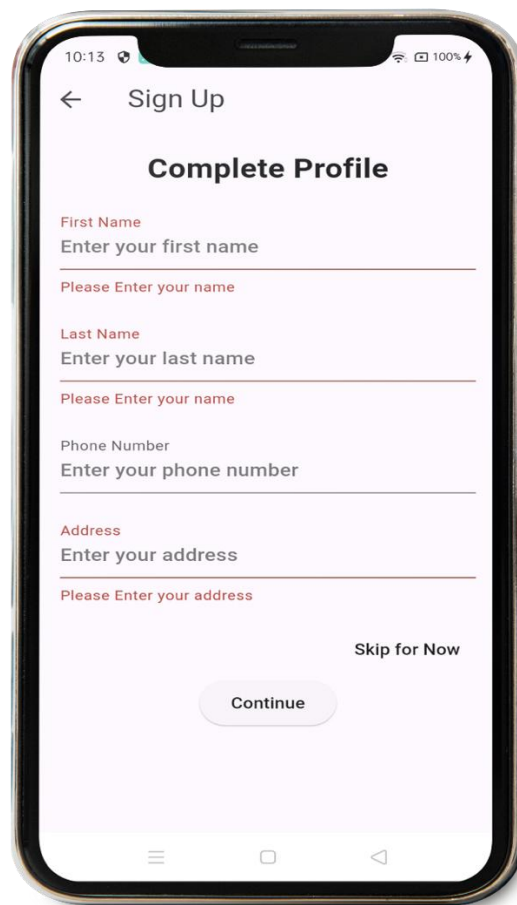


Figure 5.4 Application's Complete Profile Screen

5. Add Relatives Screen

Then you will be directed to the Add Relatives screen. This step allows the app to recognize the faces of your relatives.

- Tap on the "Upload Photo" button to add a photo of your relative. The app will read aloud the option to upload a photo.
- Enter the name of your relative. The app will prompt you to enter the name and read aloud the entered text for confirmation.
- Enter the contact number of your relative. The app will prompt you to enter the contact number and read aloud the entered text for confirmation.
- Once you have entered the details, press the "Add Relative" button.
- Press the "Continue" button to proceed.
- There is a list for the added relatives that allow you to edit or delete the relative information.

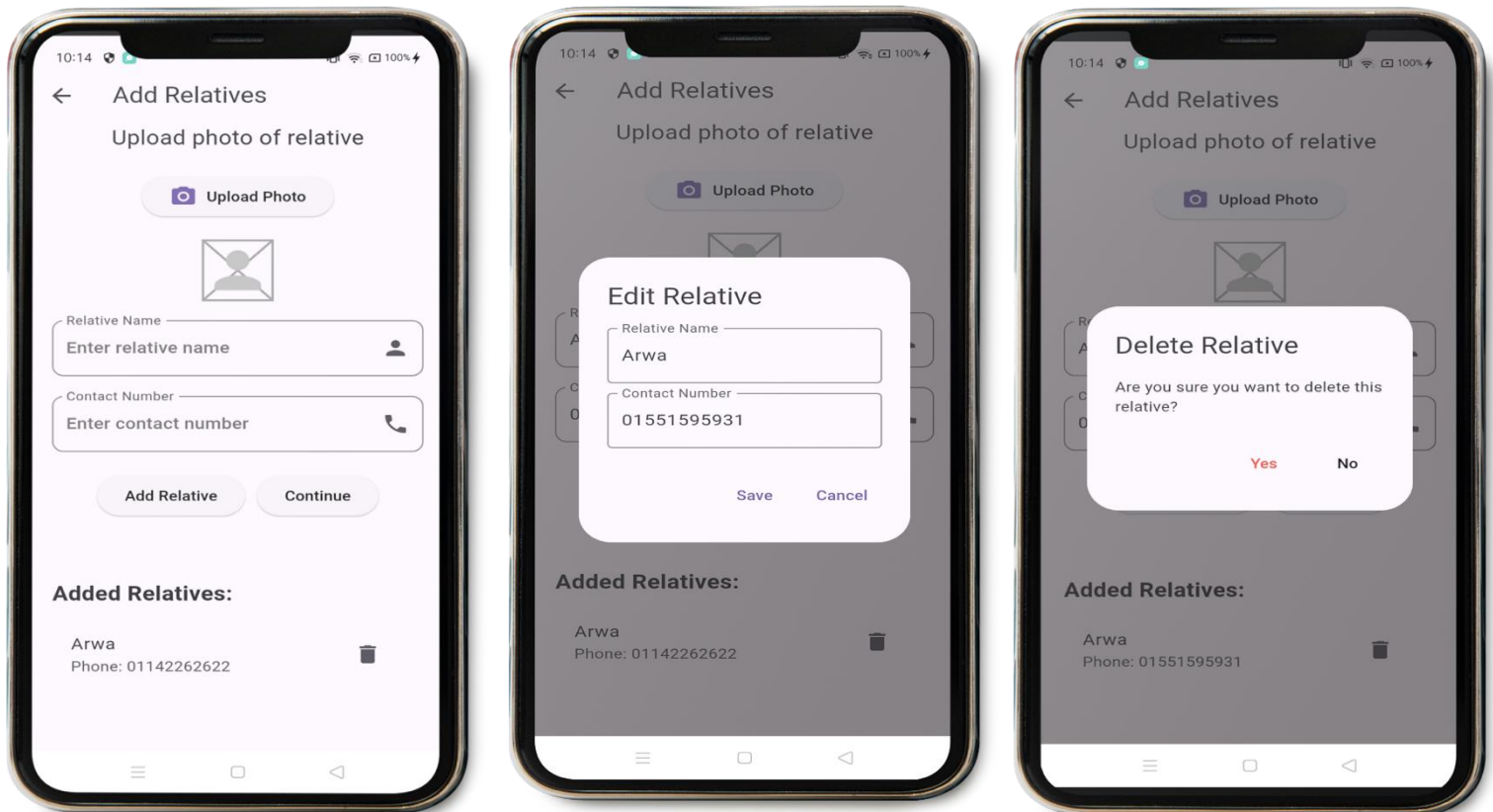


Figure 5.5 Application's Add Relative Screen

6. Main Screen

You will be directed to the main screen of the app. Here, you can access and control the primary functionalities of the Smart Eyes app.

- **Object Detection:** Toggle this switch to enable or disable object detection. The app will describe objects around you when this mode is activated.
- **Text Recognition:** Toggle this switch to enable or disable text recognition. The app will read aloud any text it detects in your surroundings.
- **Face Recognition:** Toggle this switch to enable or disable face recognition. The app will identify the faces of your registered relatives.
- **Profile Icon:** Tap on the profile icon at the top to access your profile information and settings.
- **Emergency Button:** Press the red "Emergency" button at the bottom for immediate assistance.

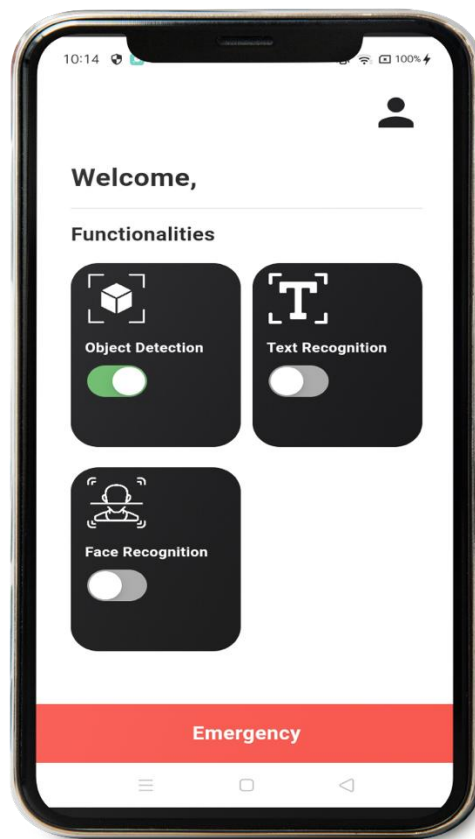


Figure 5.6 Application's Main Screen to let user choose the desired functionality

7. Emergency Screen

When you press the red "Emergency" button on the main screen, you will be directed to the Emergency Hold screen. This screen has a prominent "Emergency Hold" button.

Emergency Hold Button: Press and hold this button to initiate the emergency call process. The app will read aloud the "Emergency Hold" button prompt.

An Emergency Contacts Screen will appear after pressing and holding the "Emergency Hold" button, you will be directed to the Emergency Contacts screen. This screen displays a list of your emergency contacts phone numbers. The app will read aloud the names and numbers of your emergency contacts for confirmation, and you can choose one of the contacts to call.

These features ensure that you can quickly and easily reach your emergency contacts when necessary, providing a vital safety net.

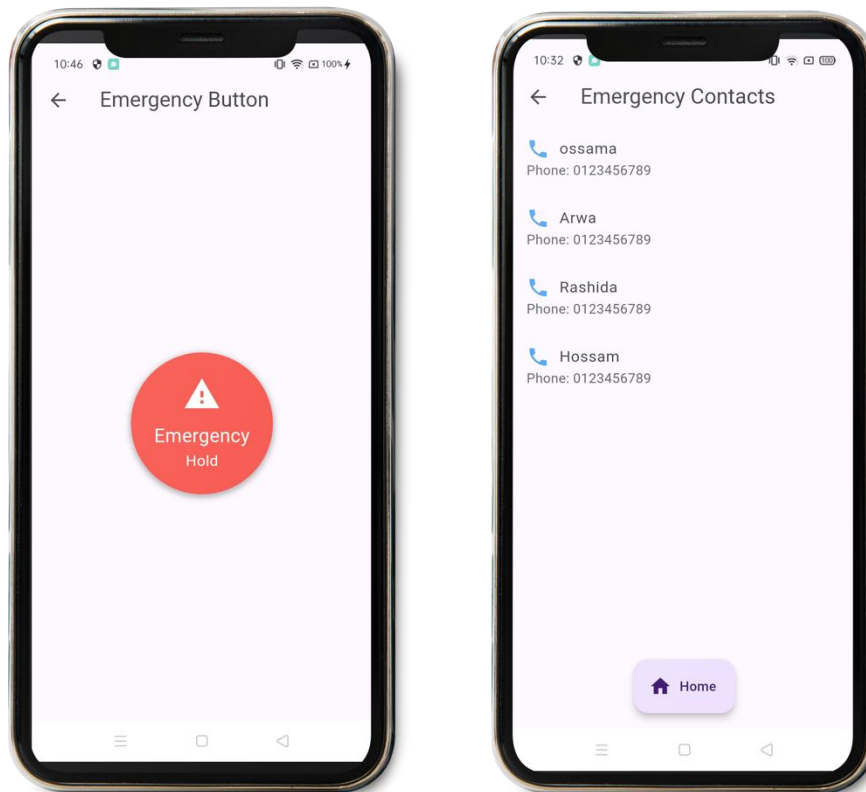


Figure 5.7 Application's Emergency Screen

8. Profile Screen

Tapping on the profile icon from the main screen directs you to the Profile screen, where you can manage your account settings.

- **Emergency Button:** Access emergency contacts.
- **Settings Button:** Configure app preferences and personalization.
- **Log Out Button:** End your current session and log out from the app.

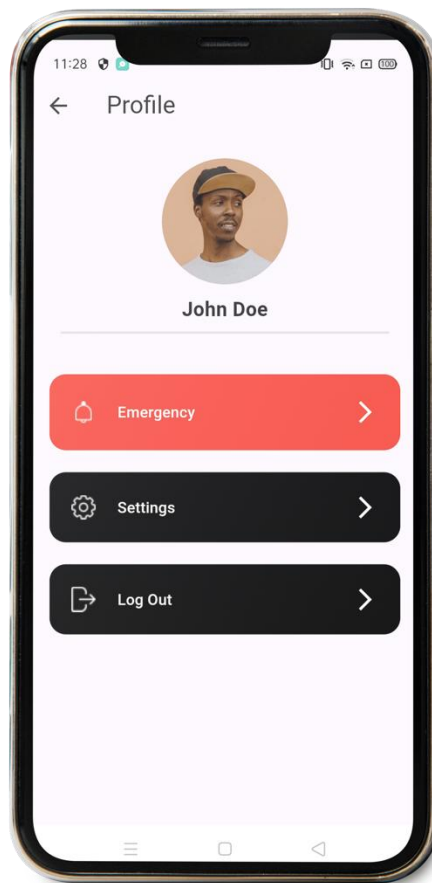


Figure 5.8 Application's Profile Screen

9. Settings Screen

When pressing the Settings button, you'll be taken to the Settings screen, which is divided into two main sections:

Account

- **Edit Profile:** Update your profile information.
- **Change Password:** Update your account password.
- **Language:** Adjust the language settings for the app.
- **Privacy and Security:** Manage your privacy and security preferences.

Notifications

- **New for You:** Toggle to receive notifications about new features or updates.
- **Account Activity:** Toggle to receive notifications about account activity.

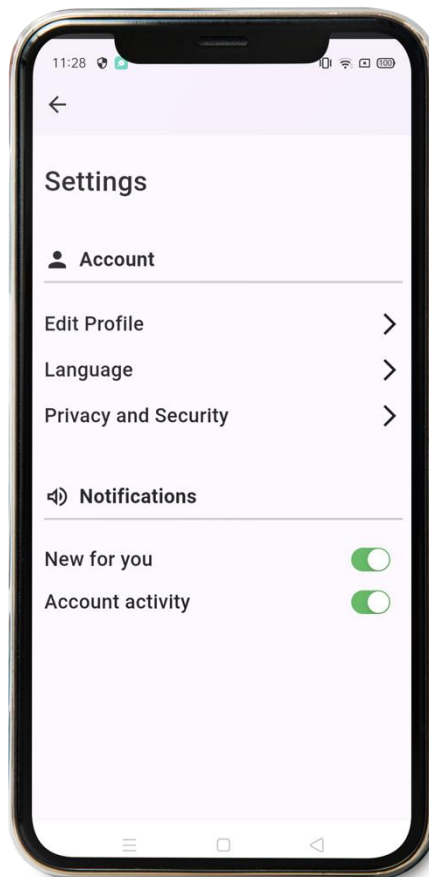


Figure 5.9 Application's Settings Screen

10. Edit Profile Screen

When you tap on the "Edit Profile" button in the Settings screen, you'll be directed to the Edit Profile screen. Here, you can update various aspects of your profile:

- **Upload Photo:** Tap here to upload or change your profile picture.
- **Edit Full Name:** Modify your first and last name as needed.
- **Edit Email:** Update your email address.
- **View/Edit Password:** Manage your account password securely.
- **Edit Location:** Update your current address.

These options allow you to customize your profile information to keep it up-to-date. Press the back arrow to return to the Settings screen when you're done.

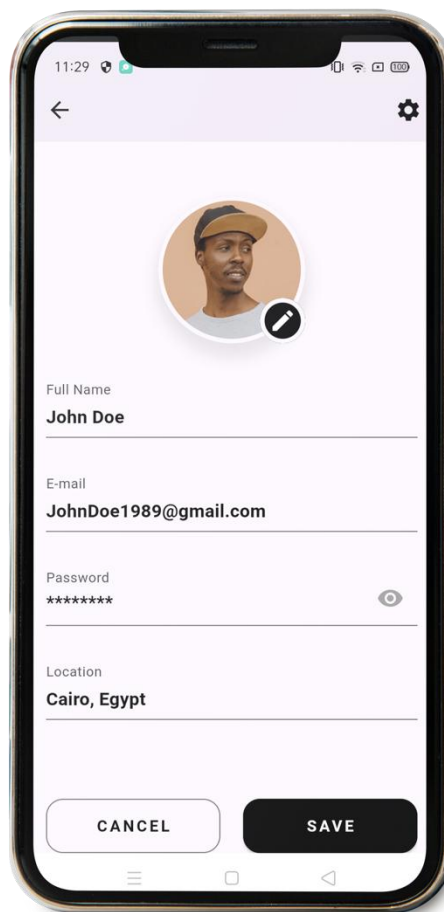


Figure 5.10 Application's Edit Profile Screen

Chapter 6

Conclusions and Future Work

6.1 Conclusions and Results

In conclusion, the Smart Eyes project successfully tackles the critical challenges faced by visually impaired individuals in recognizing and understanding their surroundings. By seamlessly integrating Optical Character Recognition (OCR), Text-to-Speech (TTS), face recognition, and object detection into a unified, user-friendly application, we aimed to significantly enhance the independence, safety, and overall quality of life for visually impaired users through real-time assistance.

Results and Key Achievements: -

OCR + TTS Mode: This mode captures frames from the camera, detects text using EasyOCR, and converts the recognized text into speech using gTTS. It empowers users to read signs, labels, and other written materials independently, facilitating greater interaction with their environment.

Face Recognition Mode: Utilizing the `face\_recognition` library, this mode captures frames, detects faces, encodes them, and compares them to stored profiles. Recognized faces are announced to the user, aiding social interactions by identifying known individuals and enhancing safety by alerting users to the presence of unfamiliar faces.

Object Detection Mode: Leveraging the YOLO model, this mode provides real-time detection and identification of objects. Detected objects are announced via gTTS, enabling users to navigate their surroundings more safely by identifying obstacles and important objects.

User Interface: The application features an accessible and intuitive interface. Each screen is designed for ease of use, incorporating voice prompts and clear navigation options to ensure seamless interaction.

Emergency Feature: An essential safety feature is the emergency button, which directs users to a screen with an "Emergency Hold" button, facilitating quick access to emergency contacts. This feature provides a critical safety net, enabling users to promptly seek help in urgent situations.

Overall, the Smart Eyes project demonstrates the effective integration of advanced computer vision and natural language processing technologies to offer comprehensive assistance to visually impaired individuals. The results affirm the efficacy of the implemented functionalities and highlight the project's potential to significantly enhance the quality of life for its users.

6.2 Future Work

While the Smart Eyes project has yielded promising results, there are several avenues for future enhancements to improve performance and introduce new functionalities.

Performance Improvements:

Optimization of Algorithms: Future work will focus on optimizing object detection and face recognition algorithms to reduce computational load and enhance real-time performance on resource-constrained devices such as wearable hardware. Implementing more efficient OCR and TTS methods can minimize latency and improve response times, making the system more responsive and user-friendly ^[11].

Enhanced Accuracy: Incorporating more advanced models for OCR, face recognition, and object detection is crucial for increasing accuracy and reliability, particularly in challenging conditions like low light or cluttered environments. Utilizing pre-trained models with higher precision and recall rates can significantly enhance detection and recognition performance ^[12].

Additional Functions:

Contextual Understanding: Developing capabilities for contextual understanding will provide more meaningful assistance. For instance, the system could describe scenes in greater detail or offer navigational directions based on recognized landmarks, thus enhancing the user's situational awareness and autonomy ^[13].

Voice Command Interface: Implementing a voice command interface will enable users to control the app and switch between modes using voice commands, facilitating hands-free operation. This feature can greatly enhance the convenience and usability of the application for visually impaired users ^[14].

Battery and Resource Management: Advanced battery and resource management techniques are essential to prolong the device's battery life during extended use. Strategies such as dynamic power management and efficient resource allocation can ensure the application remains functional for longer periods, even with intensive use.

Multilingual Support: Expanding support for multiple languages will make the application accessible to a broader audience. Incorporating multilingual capabilities ensures that users from diverse linguistic backgrounds can benefit from the application's features, enhancing its global usability.

In summary, the Smart Eyes project represents a significant advancement in leveraging technology to assist visually impaired individuals. Future work focused on performance optimization and the addition of new functionalities can further enhance the usability, efficiency, and overall experience of the application, solidifying its role as an indispensable tool for visually impaired users in their daily lives.

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436-444, 2015. doi: 10.1038/nature14539.
- [2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. of the International Conference on Learning Representations (ICLR)*, 2015.
- [3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Region-Based Convolutional Networks for Accurate Object Detection and Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 1, pp. 142-158, Jan. 2016. doi: 10.1109/TPAMI.2015.2437384.
- [4] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *Proc. of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740-755.
- [5] S. M. Anis and M. F. Anwar, "Object Recognition System for the Visually Impaired: A Deep Learning Approach using Arabic Annotation," *Electronics*, vol. 12, no. 3, p. 541, 2023.
- [6] A. Gupta, P. Singh, and R. Kumar, "Object detection and conversion of text to speech for visually impaired," in *Proc. of International Conference on Advanced Computing and Intelligent Technologies (ICACIT)*, 2022, pp. 123-130.
- [7] R. Wang, L. Zhang, and Y. Chen, "Real-Time Face Recognition System Using KPCA, LBP, and Support Vector Machine," *ResearchGate*, 2017.
- [8] Google, "Face Recognition Papers for Visually Impaired," 2023.
- [9] V. Sharma, M. K. Sharma, and A. Kumar, "Face Recognition for Visually Impaired," *International Journal of Computer Applications*, vol. 95, no. 2, pp. 23-28, 2021.
- [10] R. Gupta, S. Singh, and M. K. Gupta, "Text to Speech Conversion," *ResearchGate*, 2017.

- [11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [12] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *European Conference on Computer Vision (ECCV)*, 2016.
- [13] N. Li, S. Liu, Y. Liu, S. Zhao, and M. Liu, "Close to Human Quality TTS with Transformer," *arXiv preprint arXiv:1809.08895*, 2018.
- [14] M. Gupta, A. K. Mishra, and V. P. Jonnalagedda, "Facial recognition using deep learning," in *International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2017.